



10-x23 Generative AI

Machine Learning Department
School of Computer Science
Carnegie Mellon University

CNNs + Encoder-only Transformers + Vision Transformers

Matt Gormley
Lecture 5

Reminders

- **Homework 1: Generative Models of Text**
 - **Out: Mon, Jan 26**
 - **Due: Mon, Feb 9 at 11:59pm**

Course Staff



[Dhruv Kapur](#)



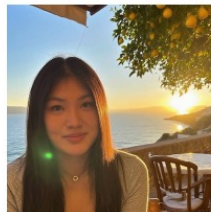
[Yuchen Shen](#)



[Bhavesh Sood](#)



[Joanna Tan](#)



[Angela Mi](#)



[James Tcheng](#)



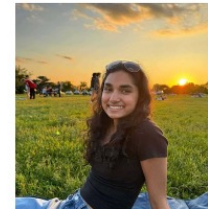
[Gopal Venki](#)



[Raj Maheshwari](#)



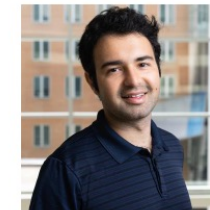
[Catherine Li](#)



[Shreya Singh](#)



[Natalie Sarabosing](#)



[Aran Nayeby](#)



[Prina Doshi](#)



[Julia Yue](#)



[Daniel Bird](#)



[Matt Gormley](#)

Recap So Far

Modern Transformers

- RoPE
 - method of relative positional embeddings
 - fundamental change to the Attention layer of a standard Transformer
 - rotates each 2D slice of a vector by some amount
- GQA
 - Transformers are computation and memory intensive
 - In GQA, we reuse some of the key/value heads, but keep the same number of query heads
 - reduces computation/memory
- Sliding Window Attention
 - key idea: change the attention mask so that each token only looks at a fixed number of tokens to the left
 - reduces computation of attention from quadratic time to linear time

CONVOLUTIONAL NEURAL NETS

A Recipe for Machine Learning

1. Given training data:

$$\mathcal{D} = \{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^N$$

2. Choose each of these:

- Decision function

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x})$$

- Loss function

$$\ell(\hat{y}, y) \in \mathbb{R}$$

3. Define goal:

$$\hat{\boldsymbol{\theta}} \approx \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \sum_{i=1}^N \ell(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)})$$

4. Train with SGD:

(take small steps
opposite the gradient)

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta_t \nabla \ell(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)})$$

A Recipe for Machine Learning

1.
 - Convolutional Neural Networks (CNNs) provide another form of **decision function**
 - Let's see what they look like...

2. Choose each of these:

- Decision function

$$\hat{y} = h_{\theta}(\mathbf{x})$$

- Loss function

$$\ell(\hat{y}, y) \in \mathbb{R}$$



4. Train with SGD:
(take small steps
opposite the gradient)

$$\theta^{(t+1)} = \theta^{(t)} - \eta_t \nabla \ell(h_{\theta}(\mathbf{x}^{(i)}), y^{(i)})$$

Convolutional Layer

CNN key idea:
Treat convolution matrix as
parameters and learn them!



Input Image

0	0	0	0	0	0	0
0	1	1	1	1	1	0
0	1	0	0	1	0	0
0	1	0	1	0	0	0
0	1	1	0	0	0	0
0	1	0	0	0	0	0
0	0	0	0	0	0	0

Learned
Convolution

θ_{11}	θ_{12}	θ_{13}
θ_{21}	θ_{22}	θ_{23}
θ_{31}	θ_{32}	θ_{33}

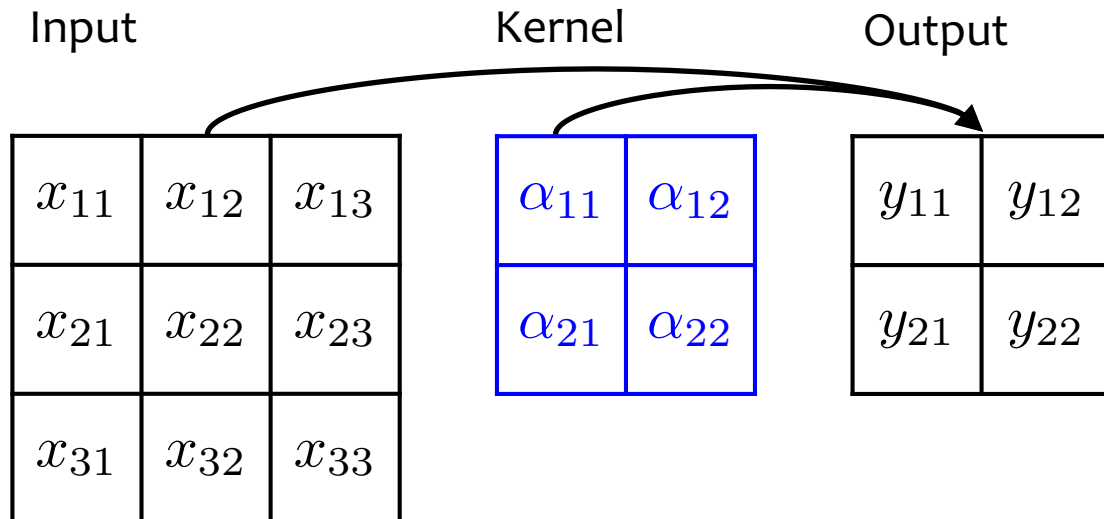
Convolved Image

.4	.5	.5	.5	.4
.4	.2	.3	.6	.3
.5	.4	.4	.2	.1
.5	.6	.2	.1	0
.4	.3	.1	0	0

2D Convolution

- Basic idea:
 - Pick a 2x2 matrix F of weights (called a kernel or convolution matrix)
 - Slide this over an image and compute the “inner product” (similarity) of F and the corresponding field of the image, and replace the pixel in the center of the field with the output of the inner product operation
- Key point:
 - Different convolutions extract different types of low-level “features” from an image
 - All that we need to vary to generate these different features is the weights of F

Example: 1 input channel, 1 output channel



$$y_{11} = \alpha_{11}x_{11} + \alpha_{12}x_{12} + \alpha_{21}x_{21} + \alpha_{22}x_{22} + \alpha_0$$

$$y_{12} = \alpha_{11}x_{12} + \alpha_{12}x_{13} + \alpha_{21}x_{22} + \alpha_{22}x_{23} + \alpha_0$$

$$y_{21} = \alpha_{11}x_{21} + \alpha_{12}x_{22} + \alpha_{21}x_{31} + \alpha_{22}x_{32} + \alpha_0$$

$$y_{22} = \alpha_{11}x_{22} + \alpha_{12}x_{23} + \alpha_{21}x_{32} + \alpha_{22}x_{33} + \alpha_0$$

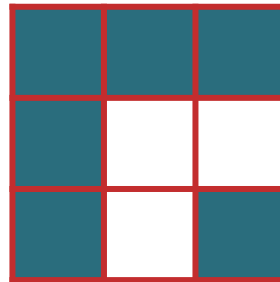
2D Convolution

- Pick a 2x2 matrix F of weights (called a kernel or convolution matrix)
- Slide this over an image and compute the “inner product” (similarity) of F and the corresponding field of the image, and replace the pixel in the center of the field with the output of the inner product operation

Input Image

0	0	0	0	0	0	0
0	1	1	1	1	1	0
0	1	0	0	1	0	0
0	1	0	1	0	0	0
0	1	1	0	0	0	0
0	1	0	0	0	0	0
0	0	0	0	0	0	0

Convolution



Convolved Image

3	2	2	3	1
2	0	2	1	0
2	2	1	0	0
3	1	0	0	0
1	0	0	0	0

Padding

Suppose you want to preserve the size of the original input image in your convolved image.

You can accomplish this by padding your input image with zeros.

Input Image

0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	1	1	1	1	1	0	0
0	0	1	0	0	1	0	0	0
0	0	1	0	1	0	0	0	0
0	0	1	1	0	0	0	0	0
0	0	1	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0
0	0	0	0	0	0	0	0	0

Identity
Convolution

0	0	0
0	1	0
0	0	0

Convolved Image

0	0	0	0	0	0	0
0	1	1	1	1	1	0
0	1	0	0	1	0	0
0	1	0	1	0	0	0
0	1	1	0	0	0	0
0	1	0	0	0	0	0
0	0	0	0	0	0	0

Stride and Downsampling

- Downsampling by averaging is a special case of convolution where the weights are fixed to a uniform distribution
- The example below uses a stride of 2
- Stride decides how many pixels we move the convolution at each step

Input Image

1	1	1	1	1	0
1	0	0	1	0	0
1	0	1	0	0	0
1	1	0	0	0	0
1	0	0	0	0	0
0	0	0	0	0	0

Convolution

$\frac{1}{4}$	$\frac{1}{4}$
$\frac{1}{4}$	$\frac{1}{4}$

Convolved Image

$\frac{3}{4}$	$\frac{3}{4}$	$\frac{1}{4}$
$\frac{3}{4}$	$\frac{1}{4}$	0
$\frac{1}{4}$	0	0

TRAINING CNNS

A Recipe for Machine Learning

1. Given training data:

$$\mathcal{D} = \{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^N$$

2. Choose each of these:

- Decision function

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x})$$

- Loss function

$$\ell(\hat{y}, y) \in \mathbb{R}$$

3. Define goal:

$$\hat{\boldsymbol{\theta}} \approx \underset{\boldsymbol{\theta}}{\operatorname{argmin}} \sum_{i=1}^N \ell(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)})$$

4. Train with SGD:

(take small steps
opposite the gradient)

$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta_t \nabla \ell(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)})$$

A Recipe for Machine Learning

1. Given training data:

$$\mathcal{D} = \{\mathbf{x}^{(i)}, y^{(i)}\}_{i=1}^N$$

2. Choose each of these:

– Decision function

$$\hat{y} = h_{\boldsymbol{\theta}}(\mathbf{x})$$

– Loss function

$$\ell(\hat{y}, y) \in \mathbb{R}$$

3. Define goal:

- Q: Now that we have the CNN as a decision function, how do we compute the gradient?
- A: Backpropagation of course!

opposite the gradient)


$$\boldsymbol{\theta}^{(t+1)} = \boldsymbol{\theta}^{(t)} - \eta_t \nabla \ell(h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)}), y^{(i)})$$

SGD for CNNs

Example: Simple CNN Architecture

Given $\mathbf{x}$, $\mathbf{y}^*$ and parameters $\boldsymbol{\theta} = [\boldsymbol{\alpha}, \boldsymbol{\beta}, \mathbf{W}]$

$$J = \ell(\mathbf{y}, \mathbf{y}^*)$$

$$\mathbf{y} = \text{softmax}(\mathbf{z}^{(5)})$$

$$\mathbf{z}^{(5)} = \text{linear}(\mathbf{z}^{(4)}, \mathbf{W})$$

$$\mathbf{z}^{(4)} = \text{relu}(\mathbf{z}^{(3)})$$

$$\mathbf{z}^{(3)} = \text{conv}(\mathbf{z}^{(2)}, \boldsymbol{\beta})$$

$$\mathbf{z}^{(2)} = \text{max-pool}(\mathbf{z}^{(1)})$$

$$\mathbf{z}^{(1)} = \text{conv}(\mathbf{x}, \boldsymbol{\alpha})$$

Algorithm 1 Stochastic Gradient Descent (SGD)

- 1: Initialize $\boldsymbol{\theta}$
 - 2: **while** not converged **do**
 - 3: Sample $i \in \{1, \dots, N\}$
 - 4: Forward: $\mathbf{y} = h_{\boldsymbol{\theta}}(\mathbf{x}^{(i)})$,
 - 5: $J(\boldsymbol{\theta}) = \ell(\mathbf{y}, \mathbf{y}^{(i)})$
 - 6: Backward: Compute $\nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
 - 7: Update: $\boldsymbol{\theta} \leftarrow \boldsymbol{\theta} - \eta \nabla_{\boldsymbol{\theta}} J(\boldsymbol{\theta})$
-

LAYERS OF A CNN

ReLU Layer

Output: $\mathbf{y} \in \mathbb{R}^K$

Forward:

$$\mathbf{y} = \sigma(\mathbf{x}), \text{ element-wise}$$
$$\sigma(a) = \max(0, a)$$

Input: $\mathbf{x} \in \mathbb{R}^K$

Input: $\frac{\partial J}{\partial \mathbf{y}} \in \mathbb{R}^K$

Backward: for each j ,

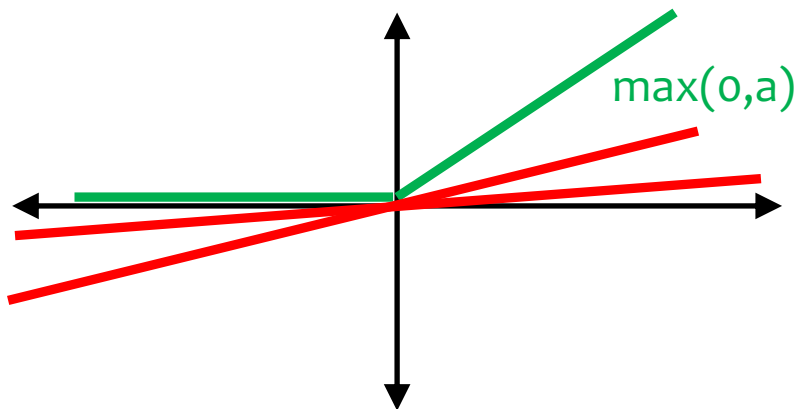
$$\frac{\partial J}{\partial x_j} = \frac{\partial J}{\partial y_j} \frac{\partial y_j}{\partial x_j}$$

where

$$\frac{\partial y_j}{\partial x_j} = \begin{cases} 1 & \text{if } x_j > 0 \\ 0 & \text{otherwise} \end{cases}$$

subderivative

Output: $\frac{\partial J}{\partial \mathbf{x}} \in \mathbb{R}^K$



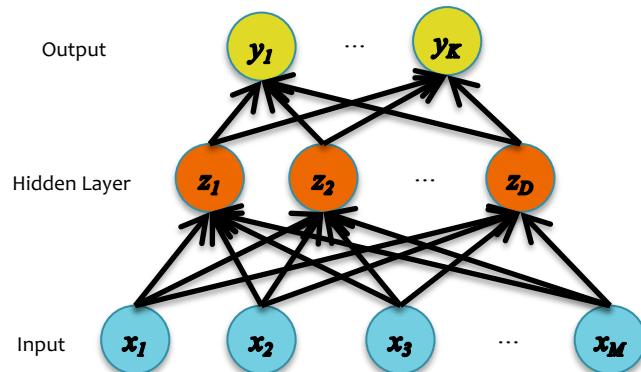
Softmax Layer

Output: $\mathbf{y} \in \mathbb{R}^K$

Forward: for each i ,

$$y_i = \frac{\exp(x_i)}{\sum_{k=1}^K \exp(x_k)}$$

Input: $\mathbf{x} \in \mathbb{R}^K$



Input: $\frac{\partial J}{\partial \mathbf{y}} \in \mathbb{R}^K$

Backward: for each j ,

$$\frac{\partial J}{\partial x_j} = \sum_{i=1}^K \frac{\partial J}{\partial y_i} \frac{\partial y_i}{\partial x_j}$$

where

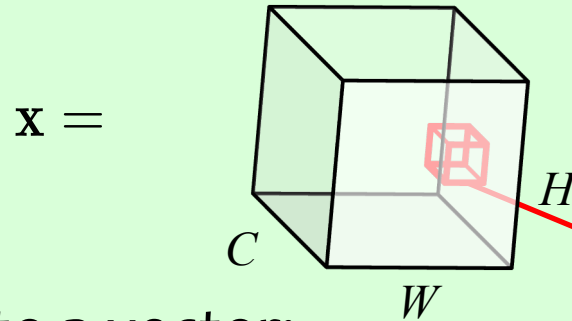
$$\frac{\partial y_i}{\partial x_j} = \begin{cases} y_i(1 - y_i) & \text{if } i = j \\ -y_i y_j & \text{otherwise} \end{cases}$$

Output: $\frac{\partial J}{\partial \mathbf{x}} \in \mathbb{R}^K$

Fully-Connected Layer (3D input)

Forward:

1. suppose input is a 3D tensor:



2. flatten out tensor into a vector:

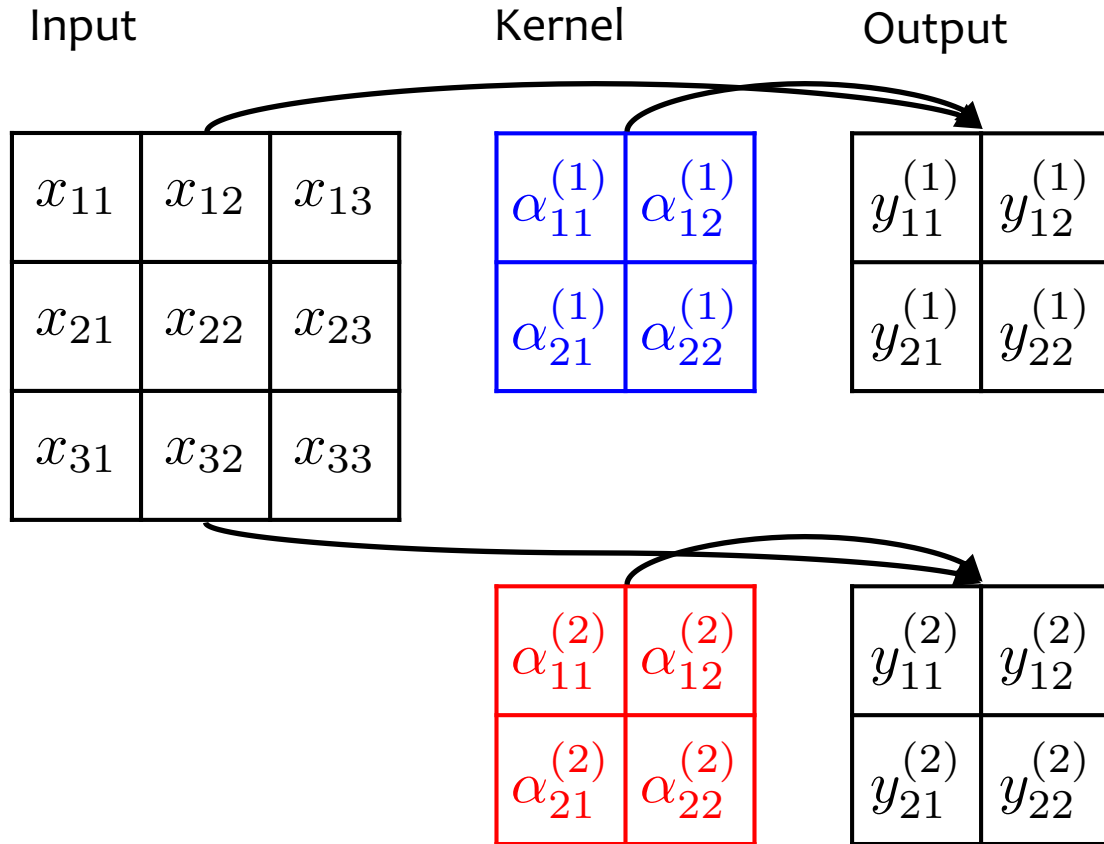
$$\hat{\mathbf{x}} = [\hat{x}_1, \dots, \hat{x}_{(C \times H \times W)}] \quad \text{where } \hat{x}_{(H \times W \times i + W \times j + k)} = x_{i,j,k}$$

3. then push that vector through a standard linear layer:

$$\mathbf{y} = \boldsymbol{\alpha}^T \hat{\mathbf{x}} + \alpha_0 \quad \text{where } \boldsymbol{\alpha} \in \mathbb{R}^{A \times B}, \quad \alpha_0 \in \mathbb{R}^B$$
$$|\hat{\mathbf{x}}| \in \mathbb{R}^A, \quad |\mathbf{y}| \in \mathbb{R}^B$$

2D Convolution

Example: 1 input channel, 2 output channels



$$y_{11}^{(1)} = \alpha_{11}^{(1)} x_{11} + \alpha_{12}^{(1)} x_{12} + \alpha_{21}^{(1)} x_{21} + \alpha_{22}^{(1)} x_{22} + \alpha_0^{(1)}$$

$$y_{12}^{(1)} = \alpha_{11}^{(1)} x_{12} + \alpha_{12}^{(1)} x_{13} + \alpha_{21}^{(1)} x_{22} + \alpha_{22}^{(1)} x_{23} + \alpha_0^{(1)}$$

$$y_{21}^{(1)} = \alpha_{11}^{(1)} x_{21} + \alpha_{12}^{(1)} x_{22} + \alpha_{21}^{(1)} x_{31} + \alpha_{22}^{(1)} x_{32} + \alpha_0^{(1)}$$

$$y_{22}^{(1)} = \alpha_{11}^{(1)} x_{22} + \alpha_{12}^{(1)} x_{23} + \alpha_{21}^{(1)} x_{32} + \alpha_{22}^{(1)} x_{33} + \alpha_0^{(1)}$$

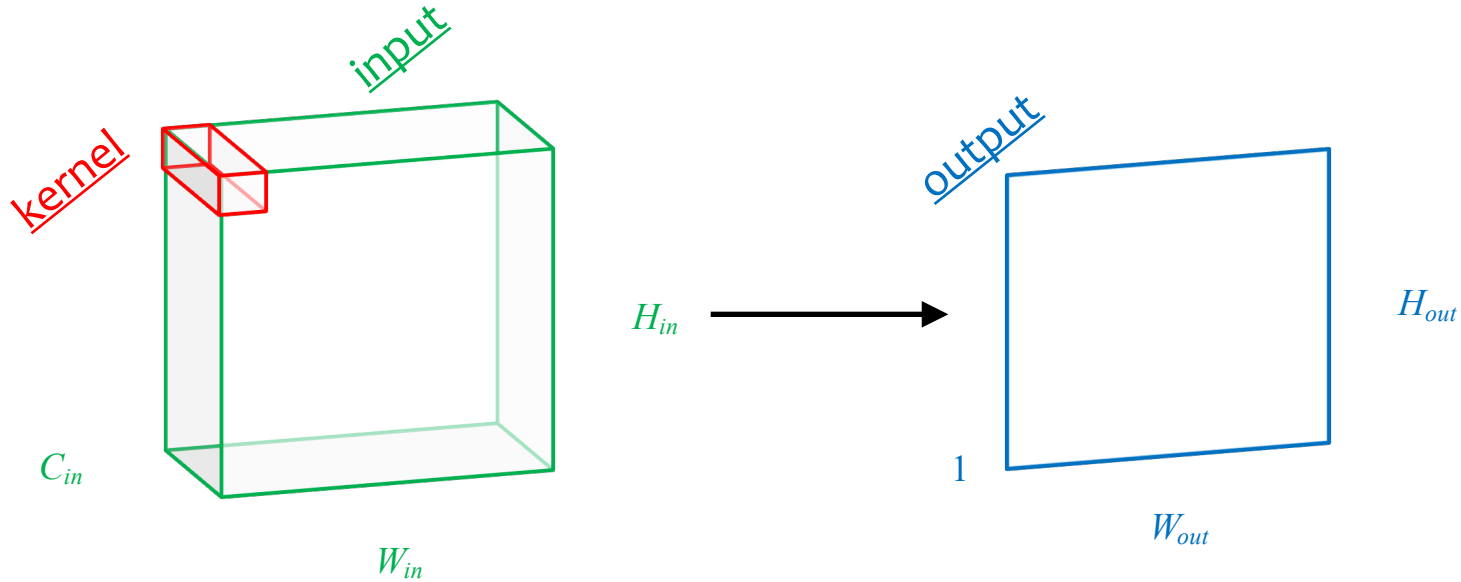
$$y_{11}^{(2)} = \alpha_{11}^{(2)} x_{11} + \alpha_{12}^{(2)} x_{12} + \alpha_{21}^{(2)} x_{21} + \alpha_{22}^{(2)} x_{22} + \alpha_0^{(2)}$$

$$y_{12}^{(2)} = \alpha_{11}^{(2)} x_{12} + \alpha_{12}^{(2)} x_{13} + \alpha_{21}^{(2)} x_{22} + \alpha_{22}^{(2)} x_{23} + \alpha_0^{(2)}$$

$$y_{21}^{(2)} = \alpha_{11}^{(2)} x_{21} + \alpha_{12}^{(2)} x_{22} + \alpha_{21}^{(2)} x_{31} + \alpha_{22}^{(2)} x_{32} + \alpha_0^{(2)}$$

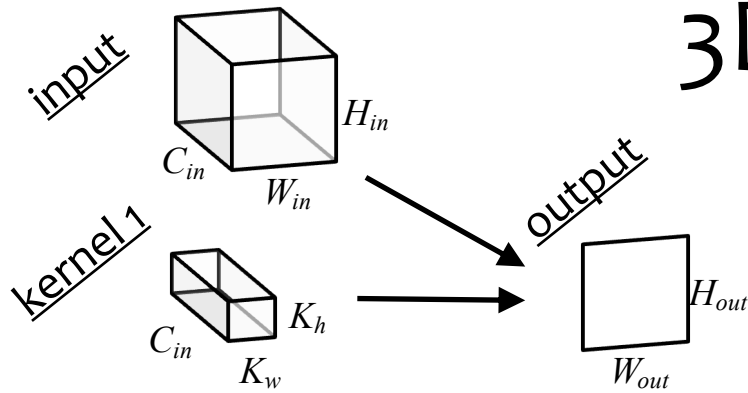
$$y_{22}^{(2)} = \alpha_{11}^{(2)} x_{22} + \alpha_{12}^{(2)} x_{23} + \alpha_{21}^{(2)} x_{32} + \alpha_{22}^{(2)} x_{33} + \alpha_0^{(2)}$$

Convolution of a Color Image

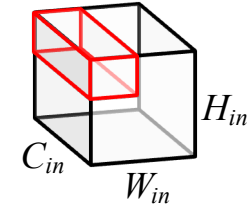


- Color images consist of 3 floats per pixel for RGB (red, green blue) color values
- The kernel must also be 3-dimensional
- input = 3x64x64
- kernel = 3x5x5
- output = 1x64x64 (assuming padding)

3D Convolutional Layer

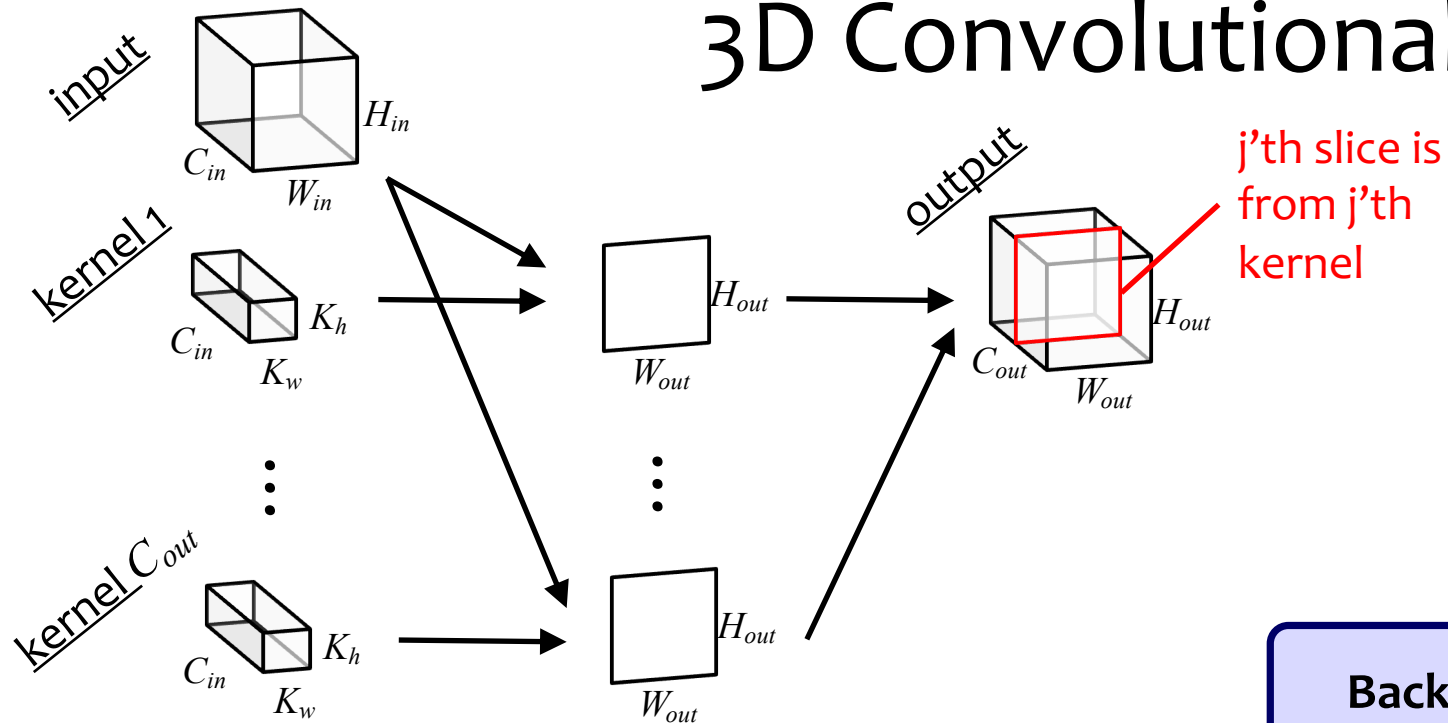


Convolution in 3D

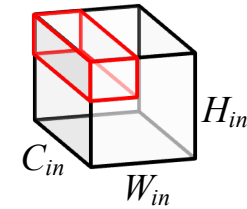


- Color images consist of 3 floats per pixel for RGB (red, green blue) color values
- Convolution must also be 3-dimensional

3D Convolutional Layer



Convolution in 3D



Forward:

$$y_{h',w'}^{(c')} = \beta^{(c')} + \sum_{c=1}^{C_{in}} \sum_{m=1}^{K_h} \sum_{n=1}^{K_w} x_{h's+m,w's+n}^{(c)} \cdot \alpha_{m,n}^{(c',c)}$$

$s \in \mathbb{Z}$ (stride)

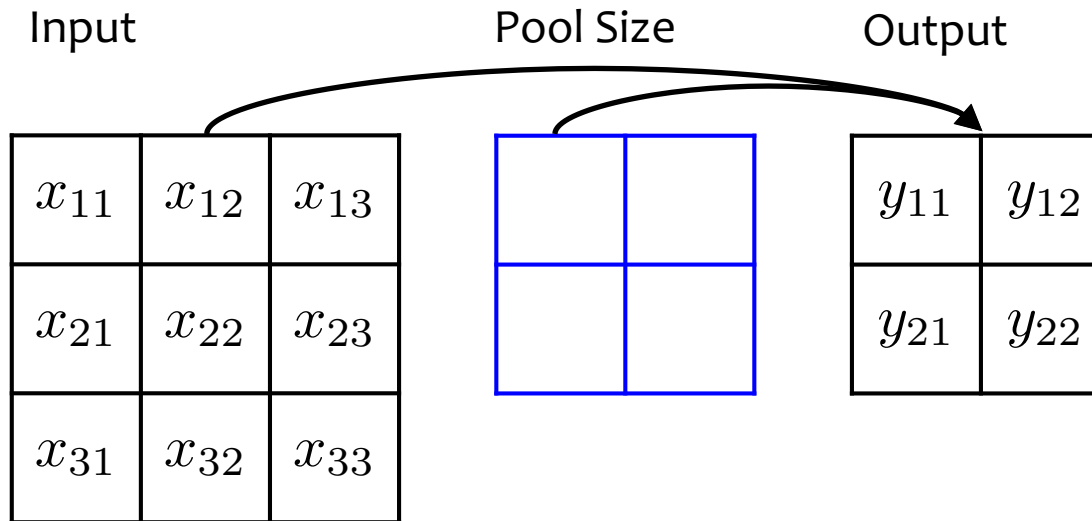
Backward:

$$\frac{\partial J}{\partial \alpha_{m,n}^{(c',c)}} = \sum_{h'=1}^{H_{out}} \sum_{w'=1}^{W_{out}} \frac{\partial J}{\partial y_{h',w'}^{(c')}} \cdot x_{h'+ms,w'+ns}^{(c)}$$

$$\frac{\partial J}{\partial \beta^{(c')}} = \sum_{h'=1}^{H_{out}} \sum_{w'=1}^{W_{out}} \frac{\partial J}{\partial y_{h',w'}^{(c')}} \cdot y_{h',w'}^{(c')}$$

Max-Pooling Layer

Example: 1 input channel, 1 output channel, stride of 1



$$y_{11} = \max(x_{11}, x_{12}, x_{21}, x_{22})$$

$$y_{12} = \max(x_{12}, x_{13}, x_{22}, x_{23})$$

$$y_{21} = \max(x_{21}, x_{22}, x_{31}, x_{32})$$

$$y_{22} = \max(x_{22}, x_{23}, x_{32}, x_{33})$$

3D Max-Pooling Layer

Output: $\mathbf{y} \in \mathbb{R}^{C \times H_{\text{out}} \times W_{\text{out}}}$

Forward:

$$y_{ij}^{(c)} = \max_{q \in \{1, \dots, K_h\}, r \in \{1, \dots, K_w\}} x_{mn}^{(c)}$$

where

$$m = s(i - 1) + q$$

$$n = s(j - 1) + r$$

Input:

$$\mathbf{x} \in \mathbb{R}^{C \times H_{\text{in}} \times W_{\text{in}}}$$

K_h, K_w (kernel size)

s (stride)

Input: $\frac{\partial J}{\partial \mathbf{y}} \in \mathbb{R}^{C \times H_{\text{out}} \times W_{\text{out}}}$

Backward:

$$\frac{\partial J}{\partial x_{mn}^{(c)}} = \sum_i \sum_j \frac{\partial J}{\partial y_{ij}^{(c)}} \frac{\partial y_{ij}^{(c)}}{\partial x_{mn}^{(c)}}$$

Output: $\frac{\partial J}{\partial \mathbf{x}} \in \mathbb{R}^{C \times H_{\text{in}} \times W_{\text{in}}}$

- $\max()$ is not differentiable, but subdifferentiable.
- There are a **set** of derivatives and we can just choose **one** for SGD

$$y = \max(a, b)$$

$$\Rightarrow \frac{dy}{da} = \frac{dy}{dy} \frac{dy}{da} \text{ where } \frac{dy}{da} = \begin{cases} 1 & \text{if } a > b \\ 0 & \text{otherwise} \end{cases}$$

CNN ARCHITECTURES

Convolutional Neural Network (CNN)

- Typical layers include:
 - Convolutional layer
 - Max-pooling layer
 - Fully-connected (Linear) layer
 - ReLU layer (or some other nonlinear activation function)
 - Softmax
- These can be arranged into arbitrarily deep topologies

Architecture #1: LeNet-5

PROC. OF THE IEEE, NOVEMBER 1998

7

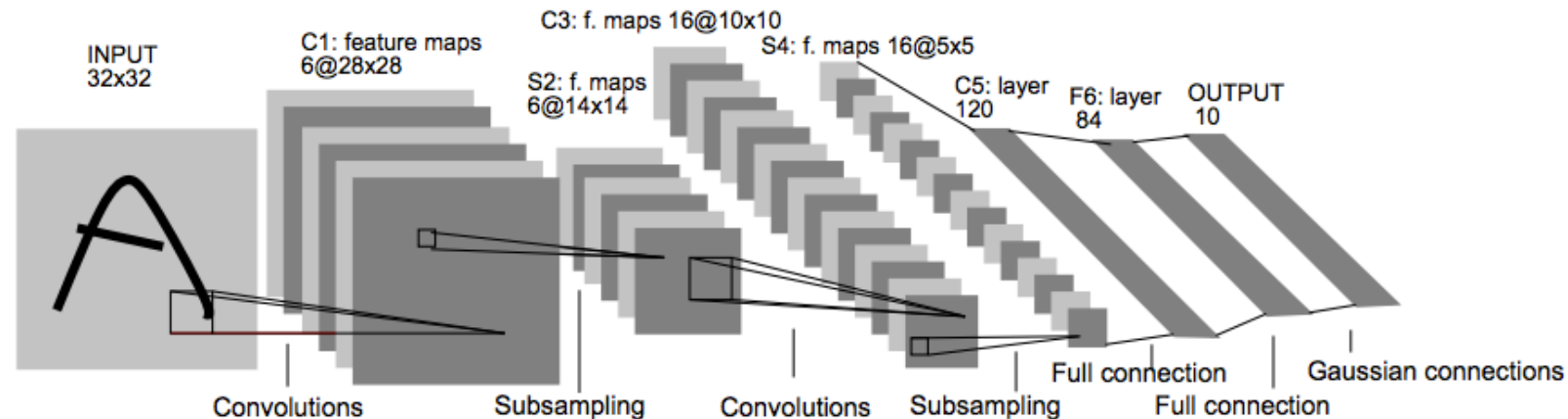


Fig. 2. Architecture of LeNet-5, a Convolutional Neural Network, here for digits recognition. Each plane is a feature map, i.e. a set of units whose weights are constrained to be identical.

Architecture #2: AlexNet

CNN for Image Classification

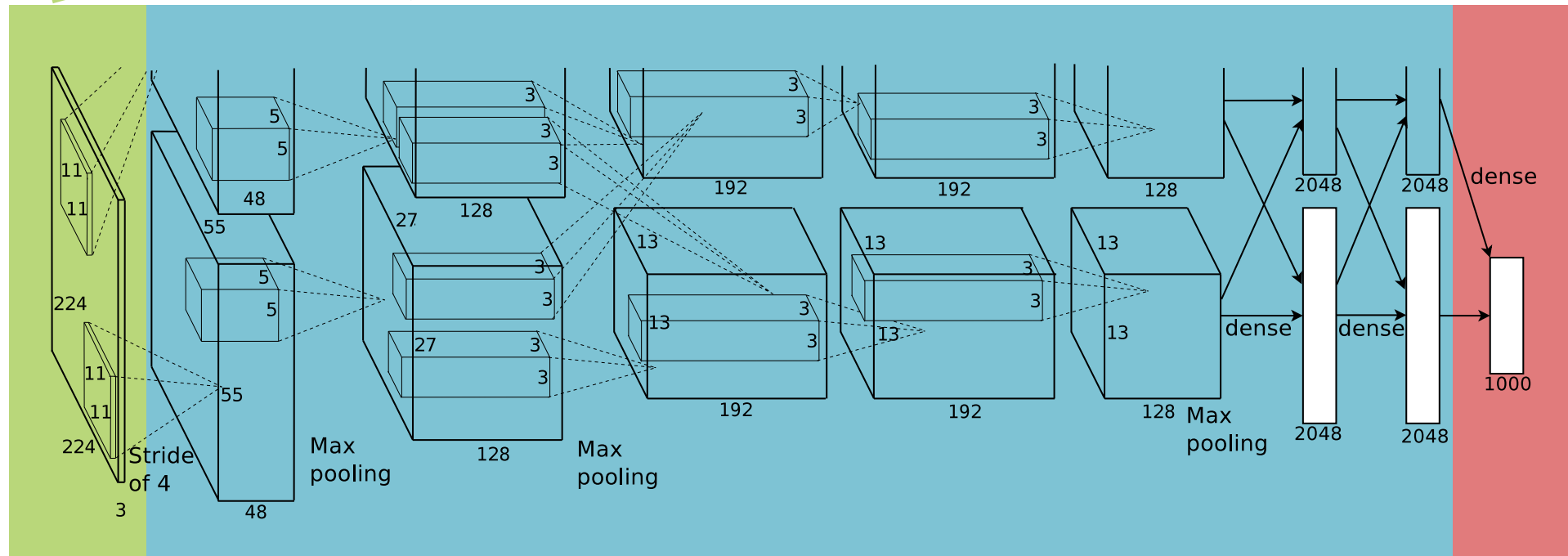
(Krizhevsky, Sutskever & Hinton, 2012)

15.3% error on ImageNet LSVRC-2012 contest

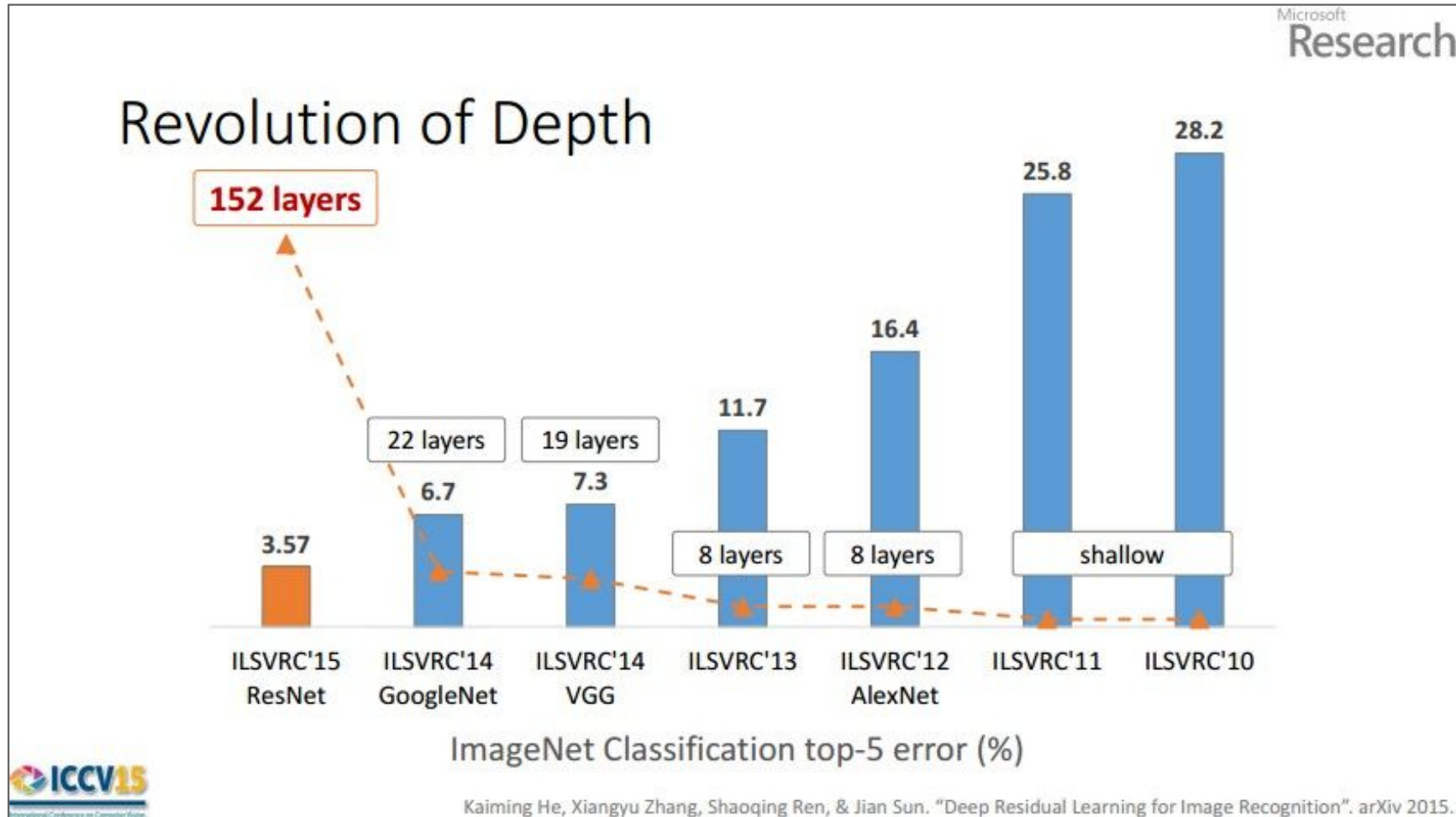
Input
image
(pixels)

- Five convolutional layers (w/max-pooling)
- Three fully connected layers

1000-way
softmax

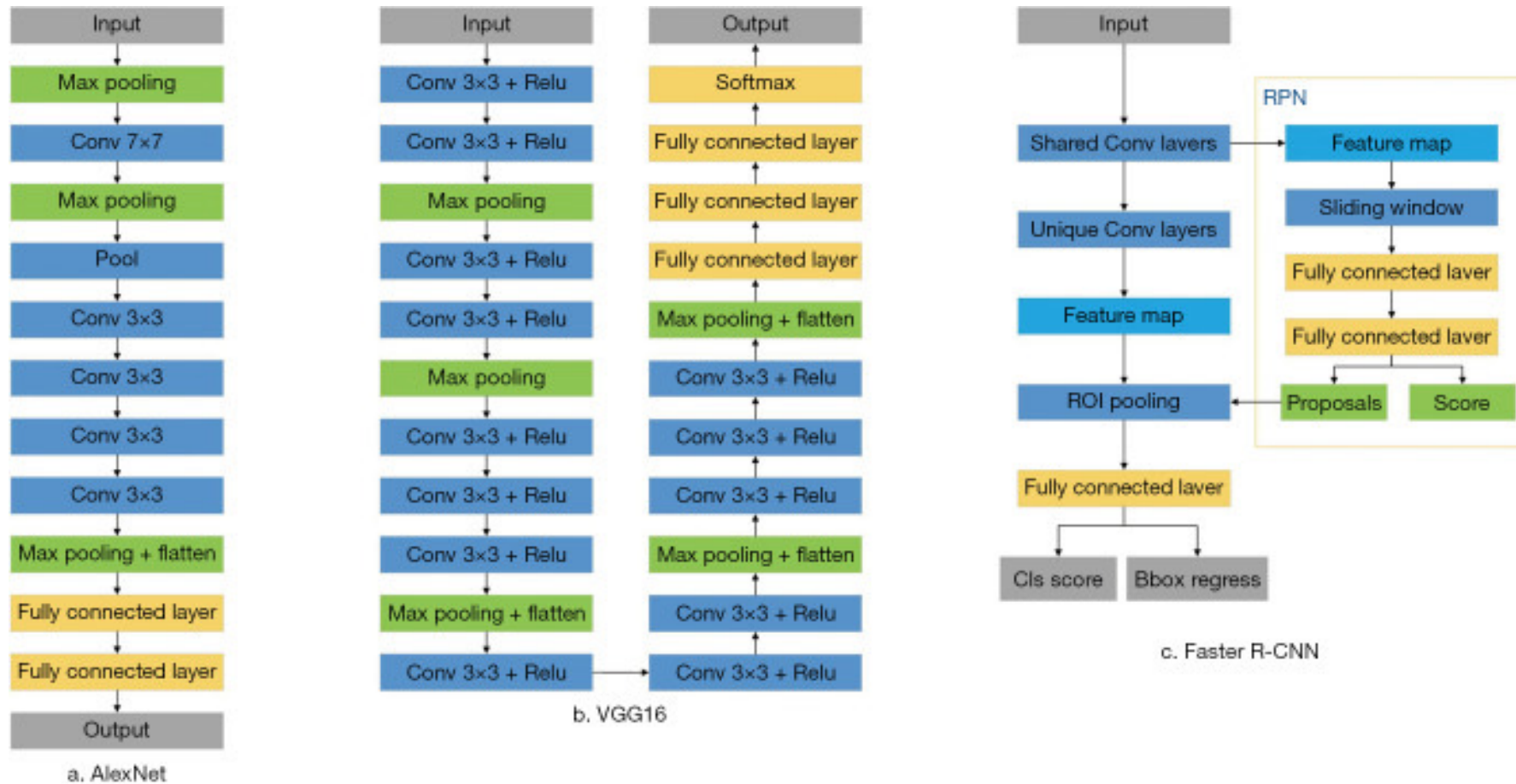


CNNs for Image Recognition



Convolutional Neural Network (CNN)

Typical Architectures



Convolutional Neural Network (CNN)

Typical Architectures

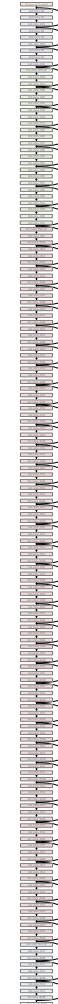
AlexNet, 8 layers
(ILSVRC 2012)



VGG, 19 layers
(ILSVRC 2014)



ResNet, 152 layers
(ILSVRC 2015)



Microsoft
Research

Convolutional Layer

For a convolutional layer, how do we pick the kernel size (aka. the size of the convolution)?

Input Image

0	0	0	0	0	0	0
0	1	1	1	1	1	0
0	1	0	0	1	0	0
0	1	0	1	0	0	0
0	1	1	0	0	0	0
0	1	0	0	0	0	0
0	0	0	0	0	0	0

2x2
Convolution

θ_{11}	θ_{12}
θ_{21}	θ_{22}

3x3
Convolution

θ_{11}	θ_{12}	θ_{13}
θ_{21}	θ_{22}	θ_{23}
θ_{31}	θ_{32}	θ_{33}

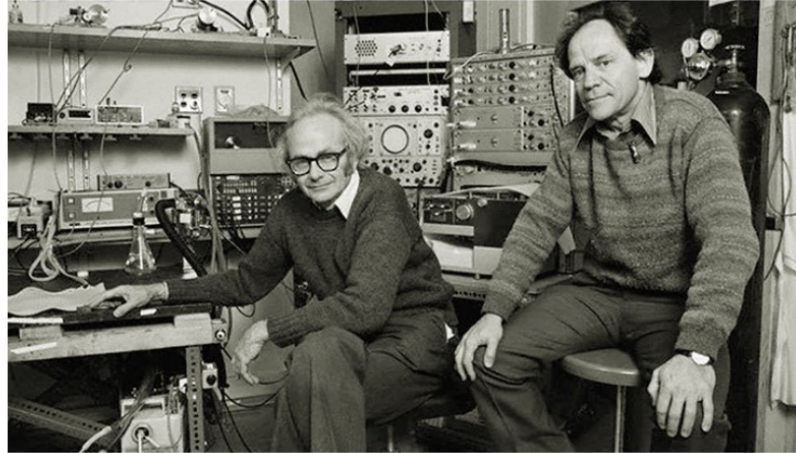
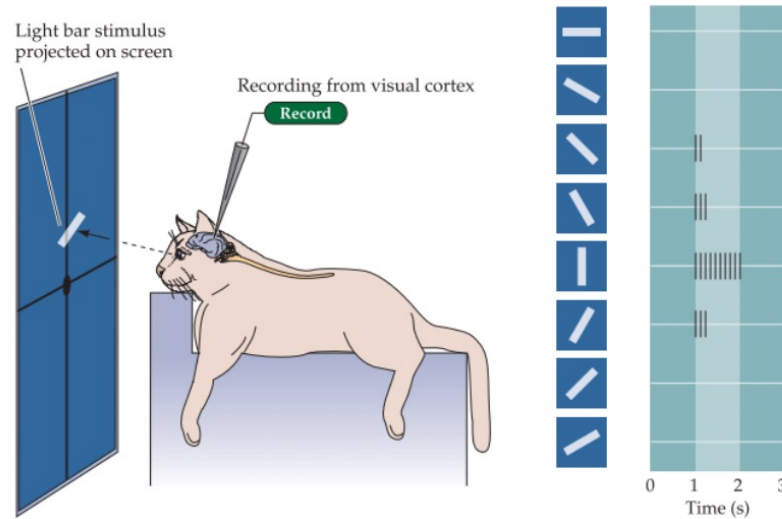
4x4
Convolution

θ_{11}	θ_{12}	θ_{13}	θ_{14}
θ_{21}	θ_{22}	θ_{23}	θ_{24}
θ_{31}	θ_{32}	θ_{33}	θ_{34}
θ_{41}	θ_{42}	θ_{43}	θ_{44}

- A small kernel can only see a very small part of the image, but is fast to compute
- A large kernel can see more of the image, but at the expense of speed

CNN's Neuroscience Origins

Re: CNN



David Hubel & Torsten Wiesel
(Nobel 1981)

- CNN operations may seem ad hoc at first, but inspired by studies in the 1960s on the cat visual cortex
- Cat is anesthetized and staring at moving bar on screen
- Hubel & Wiesel found that certain neurons would spike when the bar was upright, rather than horizontal
- This selectivity is called a receptive field
- CNN's receptive field is e.g. the 3x3 region of pixels

CNN Summary

CNNs

- Are used for all aspects of **computer vision**, and have won numerous pattern recognition competitions
- Able learn **interpretable features** at different levels of abstraction
- Typically, consist of **convolution** layers, **pooling** layers, **nonlinearities**, and **fully connected** layers

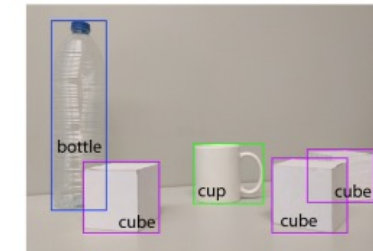
COMPUTER VISION

Common Tasks in Computer Vision

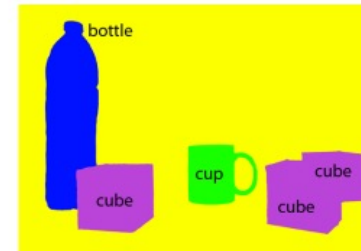
1. Image Classification
2. Image Classification + Localization
3. Human Pose Estimation
4. Semantic Segmentation
5. Object Detection
6. Instance Segmentation
7. Image Captioning
8. Image Generation



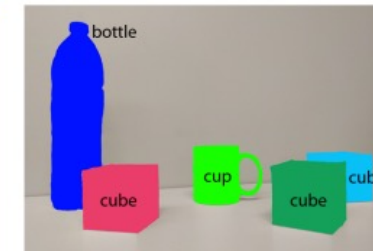
(a) Image classification



(b) Object localization



(c) Semantic segmentation



(d) Instance segmentation

Image Classification

- Given an image, predict a single label
- A multi-class classification problem

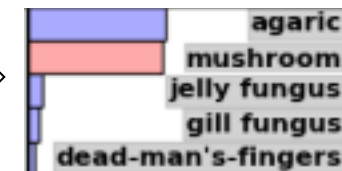
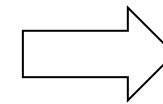
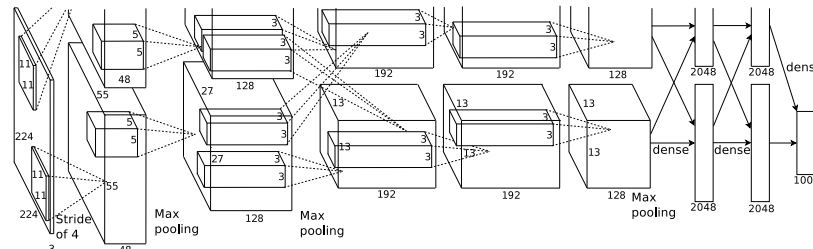
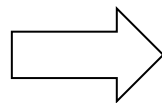
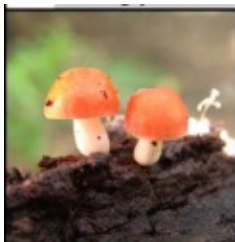
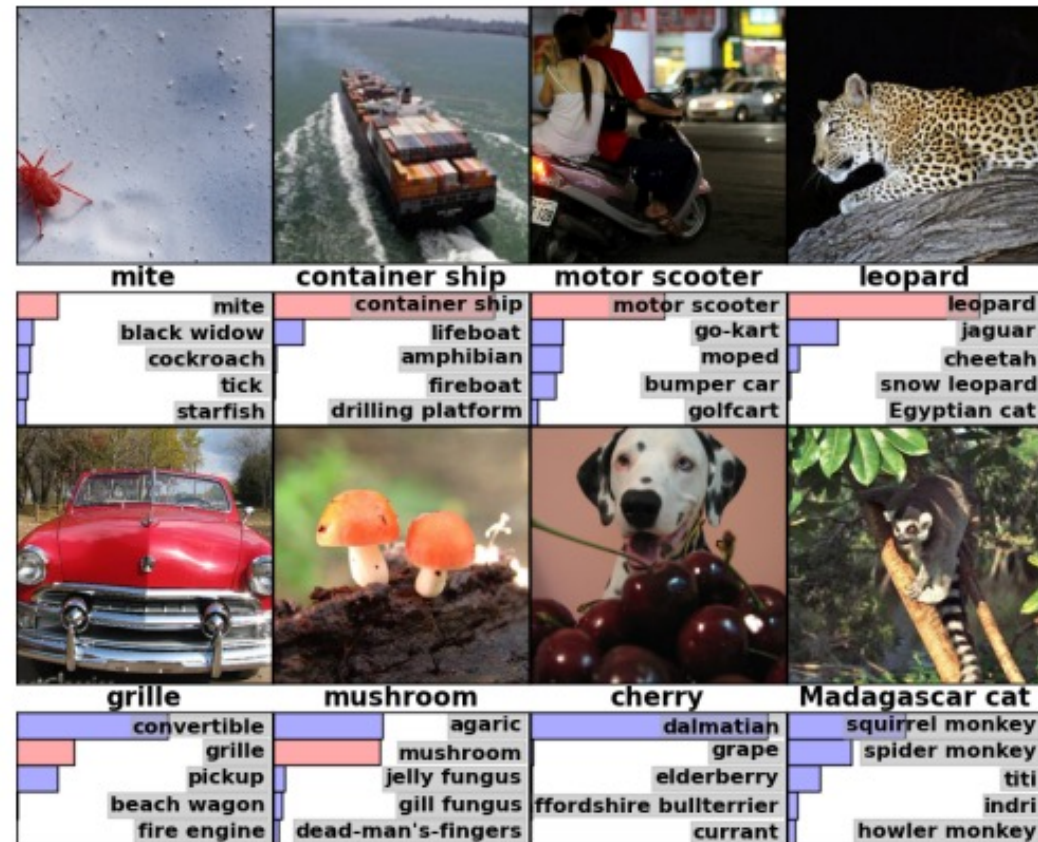


Image Classification + Localization

- Given an image, predict a single label and a bounding box for the object
- Bounding box is represented as (x, y, h, w) , position (x, y) and height/width (h, w)



Human Pose Estimation

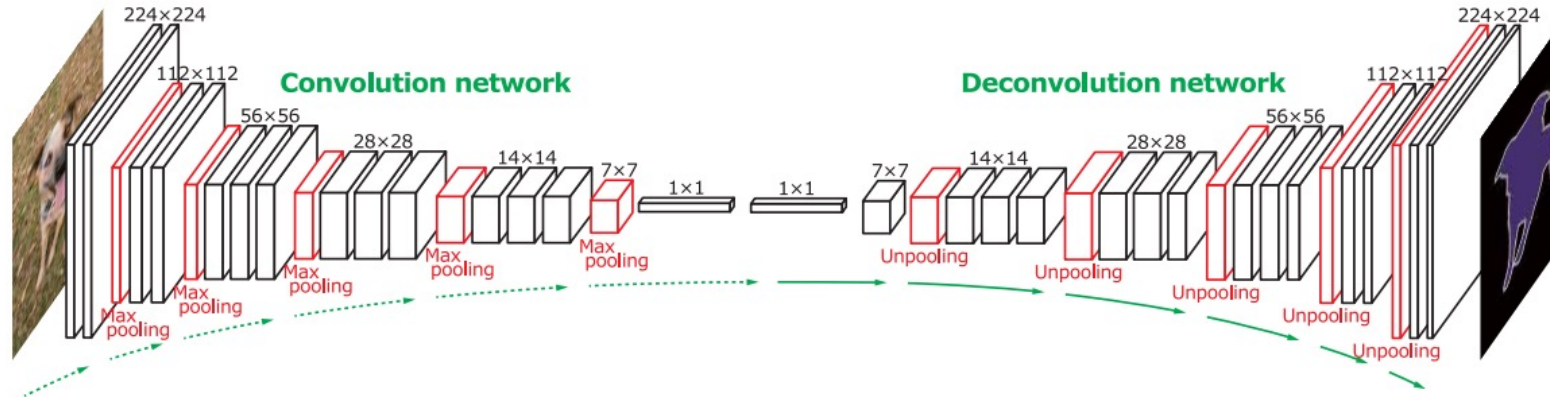
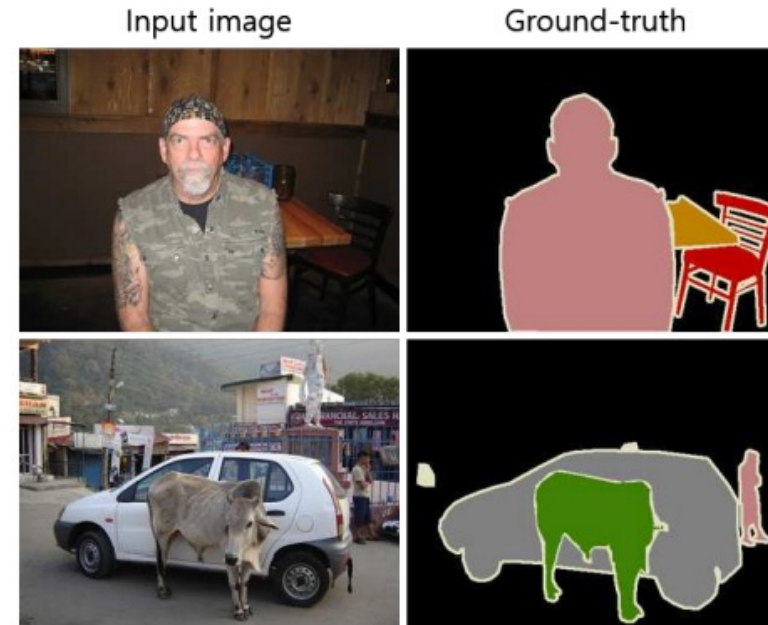


- Given an image of a human, predict the position of several keypoints (left hand, right hand, left elbow, ..., right foot)
- This is a multiple regression problem, where each keypoint has a corresponding position (x_i, y_i)



Semantic Segmentation

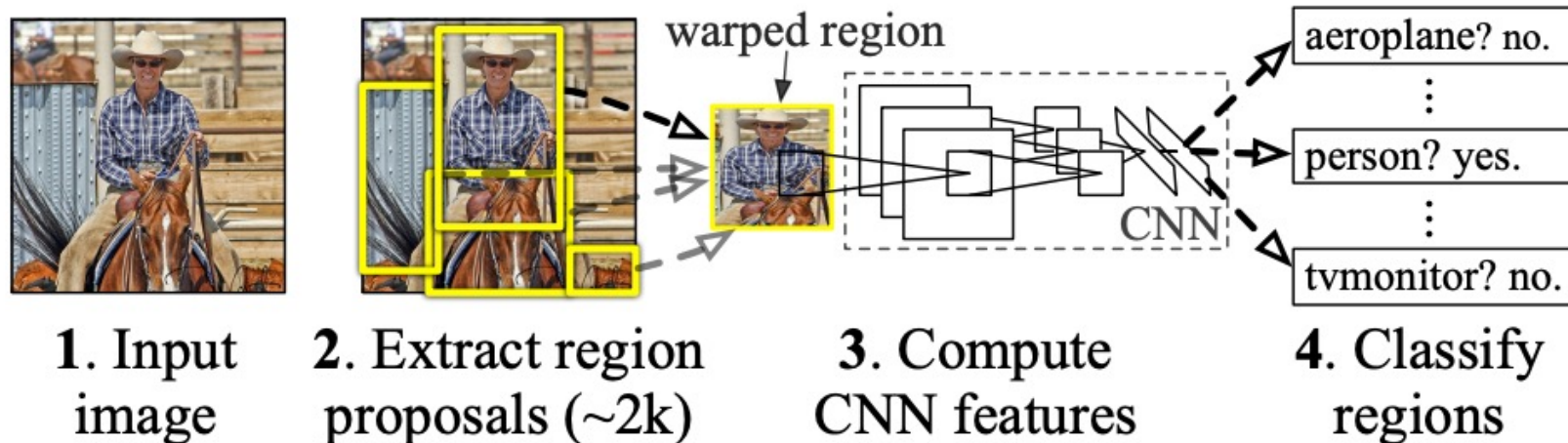
- Given an image, predict a label for every pixel in the image
- Not merely a classification problem, because there are strong correlations between pixel-specific labels



Object Detection

- Given an image, for each object predict a bounding box and a label (x, y, w, h, l)
- Example: R-CNN
 - $(x=110, y=13, w=50, h=72, l=\text{person})$
 - $(x=90, y=55, w=81, h=87, l=\text{horse})$
 - $(x=421, y=533, w=24, h=30, l=\text{chair})$
 - $(x=2, y=25, w=51, h=121, l=\text{gate})$

R-CNN: *Regions with CNN features*



Instance Segmentation

- Predict per-pixel labels as in semantic segmentation, but differentiate between different instances of the same label
- *Example:* if there are two people in the image, one person should be labeled **person-1** and one should be labeled **person-2**

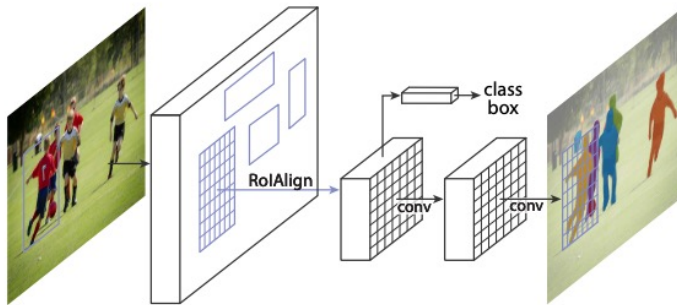
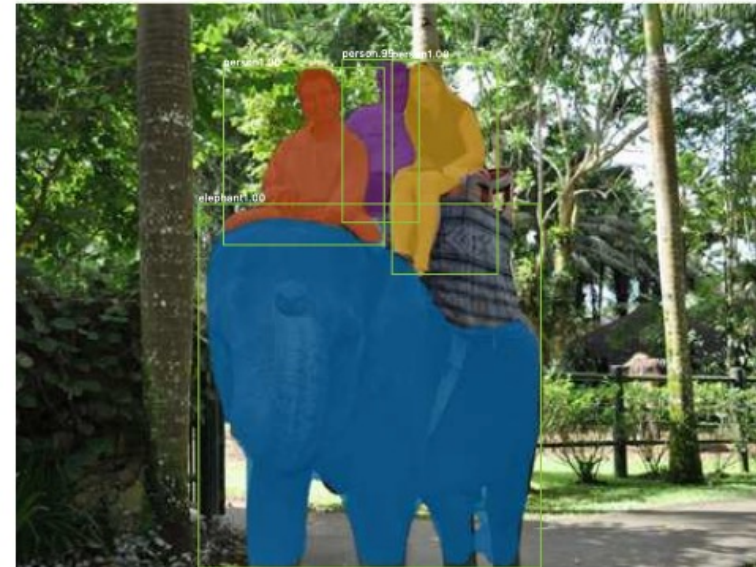
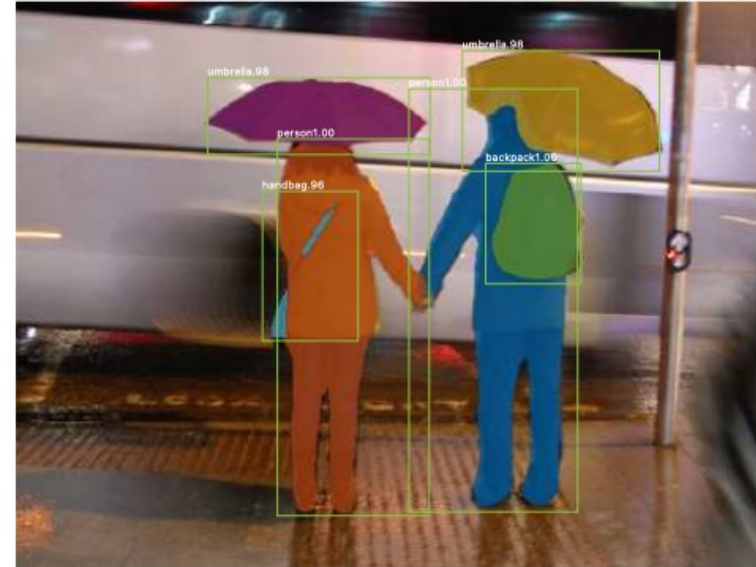
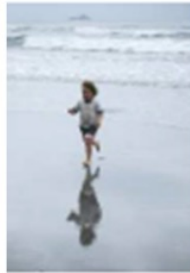


Figure 1. The **Mask R-CNN** framework for instance segmentation.

Image Captioning



Ground Truth Caption: A little boy runs away from the approaching waves of the ocean.

Generated Caption: A young boy is running on the beach.



Ground Truth Caption: A brunette girl wearing sunglasses and a yellow shirt.

Generated Caption: A woman in a black shirt and sunglasses smiles.

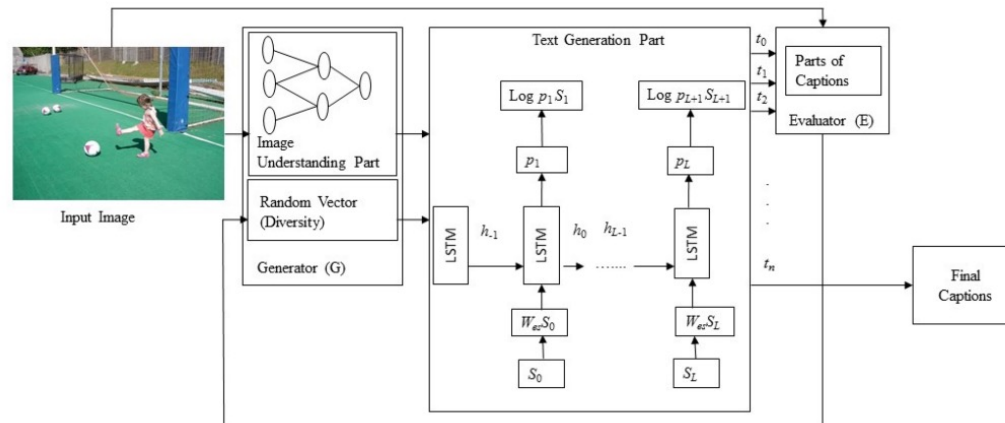


Fig. 3. A block diagram of other deep-learning-based captioning.

- Take an image as input, and generate a sentence describing it as output (i.e. the caption)
- Early methods include a deep CNN/transformer and a RNN-like language model
- (The task of *Dense Captioning* is to generate one caption per bounding box)

Image Captioning

Table 1. An Overview of the Deep-Learning-Based Approaches for Image Captioning

Reference	Image Encoder	Language Model	Category
Kiros et al. 2014 [69]	AlexNet	LBL	MS, SL, WS, EDA
Kiros et al. 2014 [70]	AlexNet, VGGNet	1. LSTM 2. SC-NLM	MS, SL, WS, EDA
Mao et al. 2014 [95]	AlexNet	RNN	MS, SL, WS
Karpathy et al. 2014 [66]	AlexNet	DTR	MS, SL, WS, EDA
Mao et al. 2015 [94]	AlexNet, VGGNet	RNN	MS, SL, WS
Chen et al. 2015 [23]	VGGNet	RNN	VS, SL, WS, EDA
Fang et al. 2015 [33]	AlexNet, VGGNet	MELM	VS, SL, WS, CA
Jia et al. 2015 [59]	VGGNet	LSTM	VS, SL, WS, EDA
Karpathy et al. 2015 [65]	VGGNet	RNN	MS, SL, WS, EDA
Vinyals et al. 2015 [142]	GoogLeNet	LSTM	VS, SL, WS, EDA
Xu et al. 2015 [152]	AlexNet	LSTM	VS, SL, WS, EDA, AB
Jin et al. 2015 [61]	VGGNet	LSTM	VS, SL, WS, EDA, AB
Wu et al. 2016 [151]	VGGNet	LSTM	VS, SL, WS, EDA, AB
Sugano et al. 2016 [129]	VGGNet	LSTM	VS, SL, WS, EDA, AB
Mathews et al. 2016 [97]	GoogLeNet	LSTM	VS, SL, WS, EDA, SC
Wang et al. 2016 [144]	AlexNet, VGGNet	LSTM	VS, SL, WS, EDA
Johnson et al. 2016 [62]	VGGNet	LSTM	VS, SL, DC, EDA
Mao et al. 2016 [92]	VGGNet	LSTM	VS, SL, WS, EDA
Wang et al. 2016 [146]	VGGNet	LSTM	VS, SL, WS, CA
Tran et al. 2016 [135]	ResNet	MELM	VS, SL, WS, CA
Ma et al. 2016 [90]	AlexNet	LSTM	VS, SL, WS, CA
You et al. 2016 [156]	GoogLeNet	RNN	VS, SL, WS, EDA, SCB
Yang et al. 2016 [153]	VGGNet	LSTM	VS, SL, DC, EDA
Anne et al. 2016 [6]	VGGNet	LSTM	VS, SL, WS, CA, NOB
Yao et al. 2017 [155]	GoogLeNet	LSTM	VS, SL, WS, EDA, SCB
Lu et al. 2017 [88]	ResNet	LSTM	VS, SL, WS, EDA, AB
Chen et al. 2017 [21]	VGGNet, ResNet	LSTM	VS, SL, WS, EDA, AB
Gan et al. 2017 [41]	ResNet	LSTM	VS, SL, WS, CA, SCB
Pedersoli et al. 2017 [112]	VGGNet	RNN	VS, SL, WS, EDA, AB
Ren et al. 2017 [119]	VGGNet	LSTM	VS, ODL, WS, EDA
Park et al. 2017 [111]	ResNet	LSTM	VS, SL, WS, EDA, AB
Wang et al. 2017 [148]	ResNet	LSTM	VS, SL, WS, EDA
Tavakoli et al. 2017 [134]	VGGNet	LSTM	VS, SL, WS, EDA, AB
Liu et al. 2017 [84]	VGGNet	LSTM	VS, SL, WS, EDA, AB
Gan et al. 2017 [39]	ResNet	LSTM	VS, SL, WS, EDA, SC
Dai et al. 2017 [26]	VGGNet	LSTM	VS, ODL, WS, EDA
Shetty et al. 2017 [126]	GoogLeNet	LSTM	VS, ODL, WS, EDA
Liu et al. 2017 [85]	Inception-V3	LSTM	VS, ODL, WS, EDA
Gu et al. 2017 [51]	VGGNet	1. Language CNN 2. LSTM	VS, SL, WS, EDA
Yao et al. 2017 [154]	VGGNet	LSTM	VS, SL, WS, CA, NOB

(Continued)

- Take an image as input, and generate a sentence describing it as output (i.e. the caption)
- Early methods included a deep CNN/transformer and a RNN-like language model
- (The task of *Dense Captioning* is to generate one caption per bounding box)

Medical Image Analysis

Notice that **most** of these tasks are structured prediction problems, not merely classification

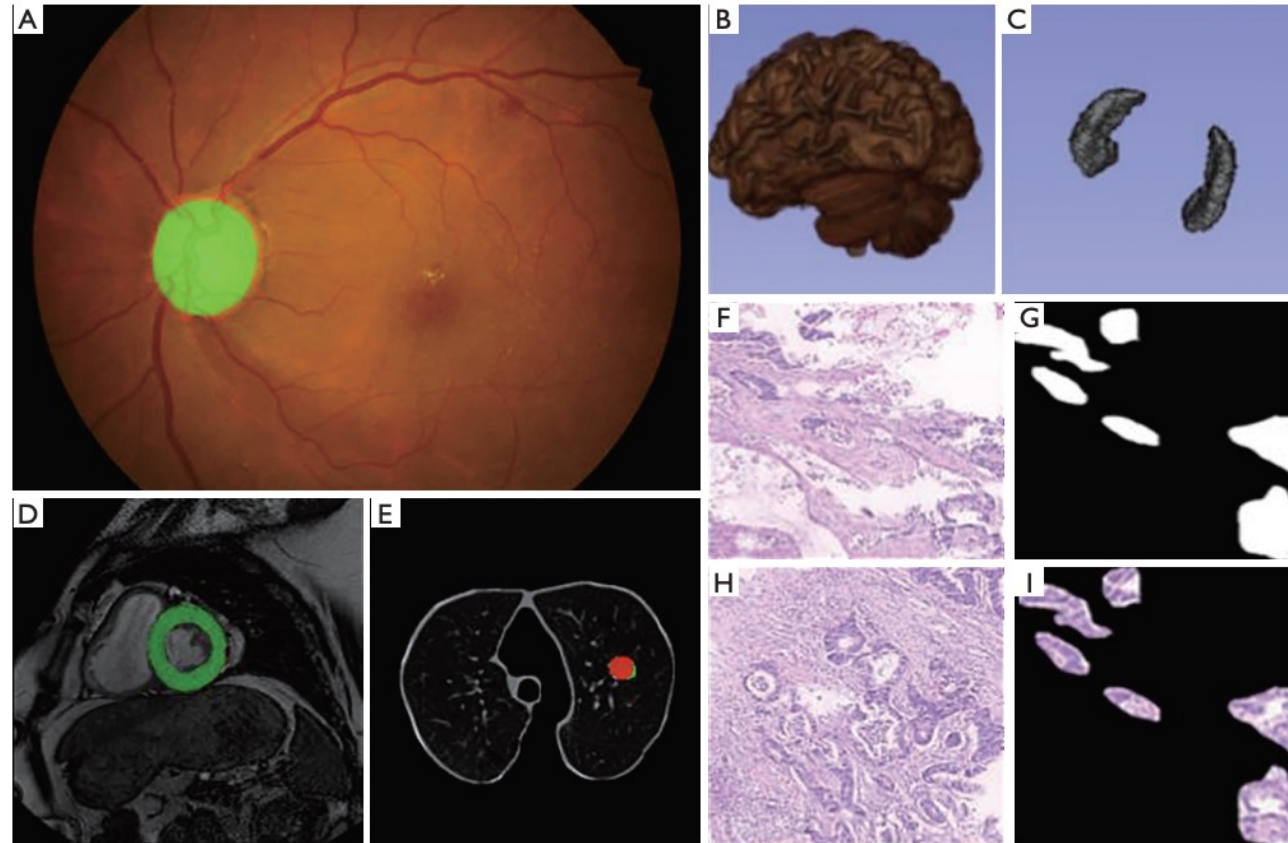


Figure 2 Deep learning application in medical image analysis. (A) Fundus detection; (B,C) hippocampus segmentation; (D) left ventricular segmentation; (E) pulmonary nodule classification; (F,G,H,I) gastric cancer pathology segmentation. The staining method is H&E, and the magnification is $\times 40$.

(aka. BERT-style models)

TRANSFORMER (ENCODER ONLY VERSION)

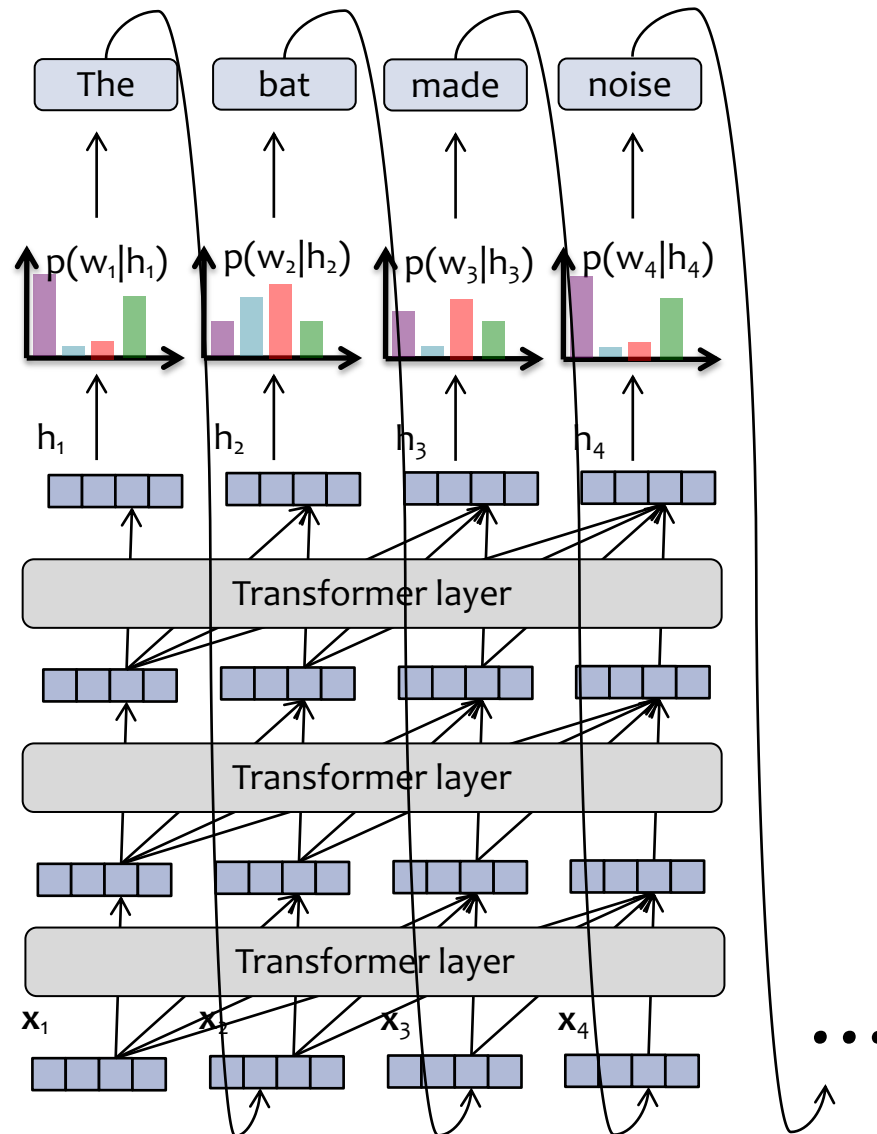
Autoregressive Language Model

Definition: An *autoregressive language model* defines a probability distribution over sequences $\mathbf{x}_{1:T}$ of the form:

$$p(\mathbf{x}_{1:T}) = \prod_{t=1}^T p(x_t \mid x_1, \dots, x_{t-1})$$

Decoder-only Transformer

Also called a
**Transformer
language model**



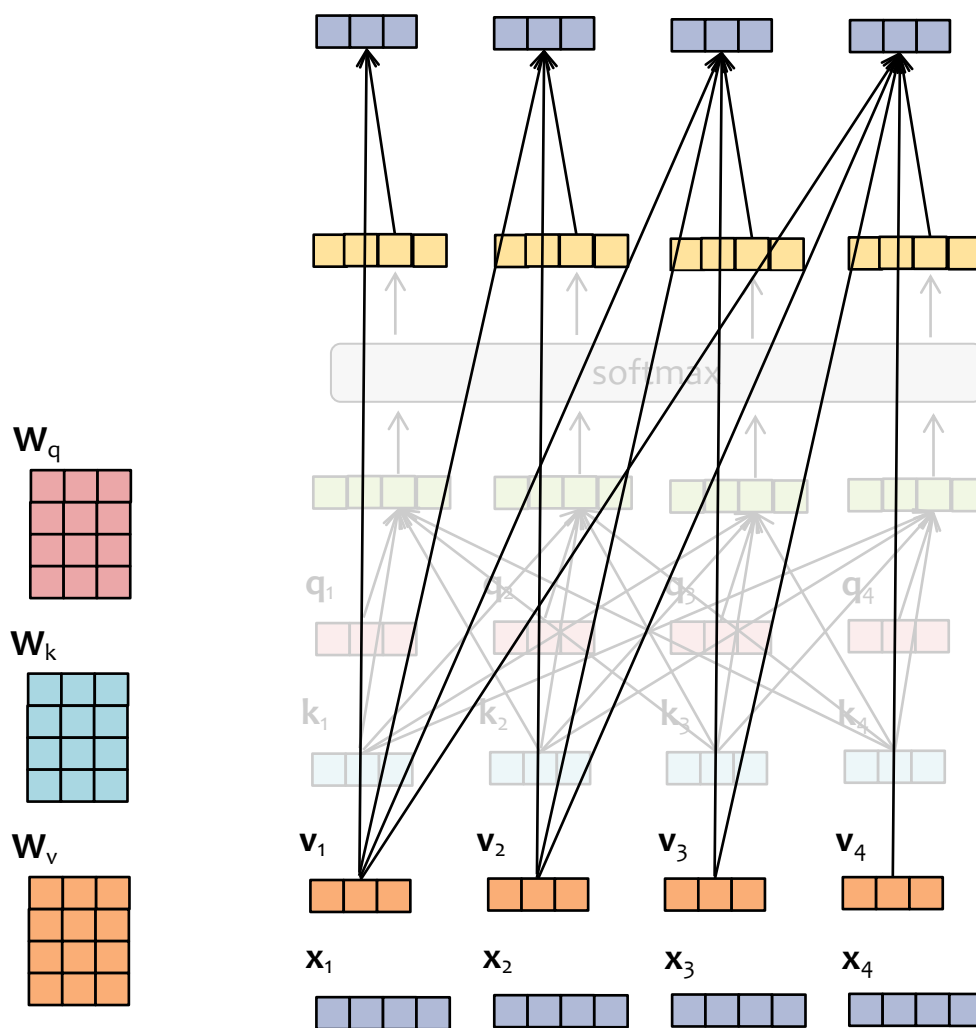
Each layer of a Transformer LM consists of several **sublayers**:

1. causal attention
2. feed-forward neural network
3. layer normalization
4. residual connections

Each hidden vector looks back at the hidden vectors of the **current and previous timesteps in the previous layer**.

The language model part is just like an RNN-LM.

Single-Headed (Causal) Attention



$$X' = AV = \text{softmax}(QK^T / \sqrt{d_k} + M)V$$

$$A_{\text{causal}} = \text{softmax}(S + M)$$

$$S = QK^T / \sqrt{d_k}$$

$$Q = XW_q$$

$$K = XW_k$$

$$V = XW_v$$

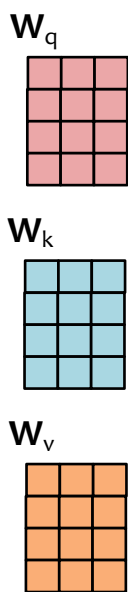
$$X = [x_1, \dots, x_4]^T$$

In practice, the attention weights are computed for all time steps T , then we mask out (by setting to $-\infty$) all the inputs to the softmax that are for the timesteps to the right of the query

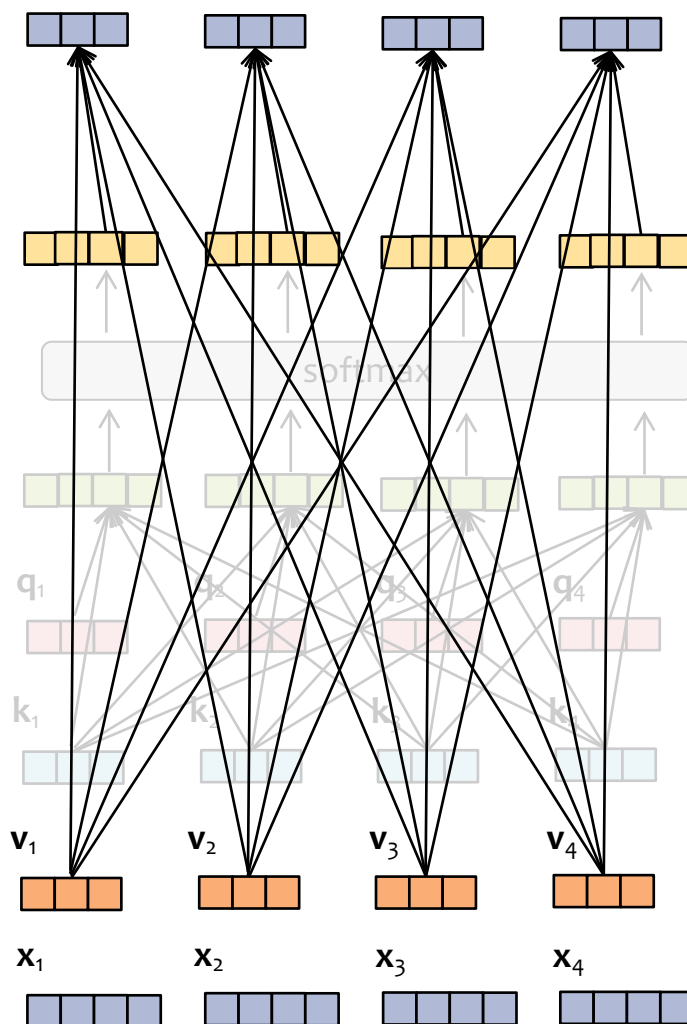
$$M = \begin{bmatrix} 0 & -\infty & -\infty & -\infty \\ 0 & 0 & -\infty & -\infty \\ 0 & 0 & 0 & -\infty \\ 0 & 0 & 0 & 0 \end{bmatrix}$$

Holy cow, that's a lot of new arrows...

- Do we *always* want/need all of those?
No!
- Do we sometimes want/need all of those?
Yes!



Single-Headed Attention



$$\mathbf{X}' = \mathbf{A}\mathbf{V} = \text{softmax}(\mathbf{Q}\mathbf{K}^T / \sqrt{d_k})\mathbf{V}$$

$$\mathbf{A} = \text{softmax}(\mathbf{S})$$

$$\mathbf{S} = \mathbf{Q}\mathbf{K}^T / \sqrt{d_k}$$

$$\mathbf{Q} = \mathbf{X}\mathbf{W}_q$$

$$\mathbf{K} = \mathbf{X}\mathbf{W}_k$$

$$\mathbf{V} = \mathbf{X}\mathbf{W}_v$$

$$\mathbf{X} = [\mathbf{x}_1, \dots, \mathbf{x}_4]^T$$

So what is this model?

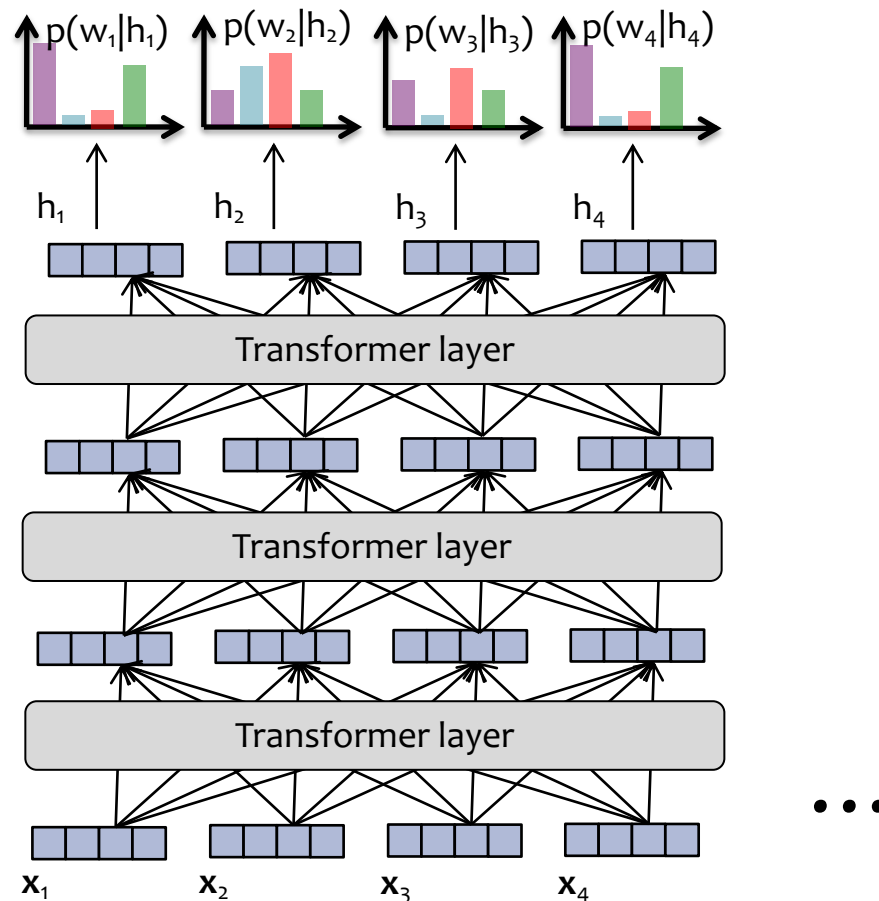
- Notice that each token can attend to all other tokens on the left and right
- It is **not** an autoregressive language model

Encoder-only Transformer

Each layer of an encoder-only Transformer consists of several **sublayers**:

1. non-causal attention
2. feed-forward neural network
3. layer normalization
4. residual connections

Each hidden vector looks at the the hidden vectors of **all timesteps in the previous layer**.



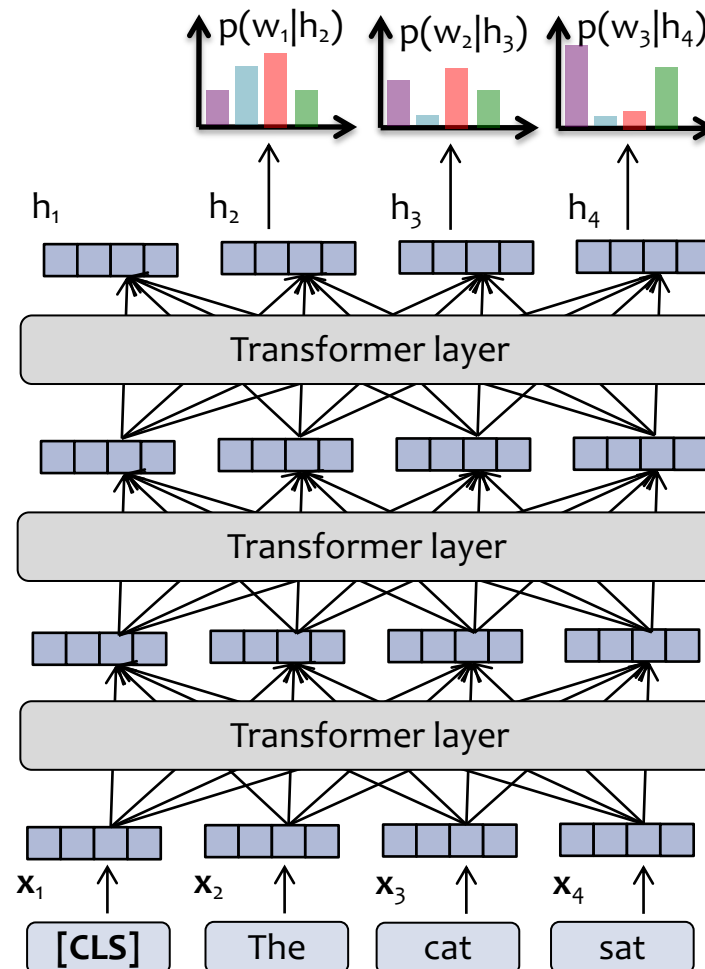
Okay, but how would we train one of these things?

Encoder-only Transformer

BERT popularized this encoder-only Transformer architecture and style of pretraining

MLM Pretraining:

- Rather than trying to predict the next word from the previous ones...
- ...mask out a word (or a few words) and predict the missing words from the remaining ones



Each layer of an encoder-only Transformer consists of several **sublayers**:

1. non-causal attention
2. feed-forward neural network
3. layer normalization
4. residual connections

Each hidden vector looks at the the hidden vectors of **all timesteps in the previous layer**.

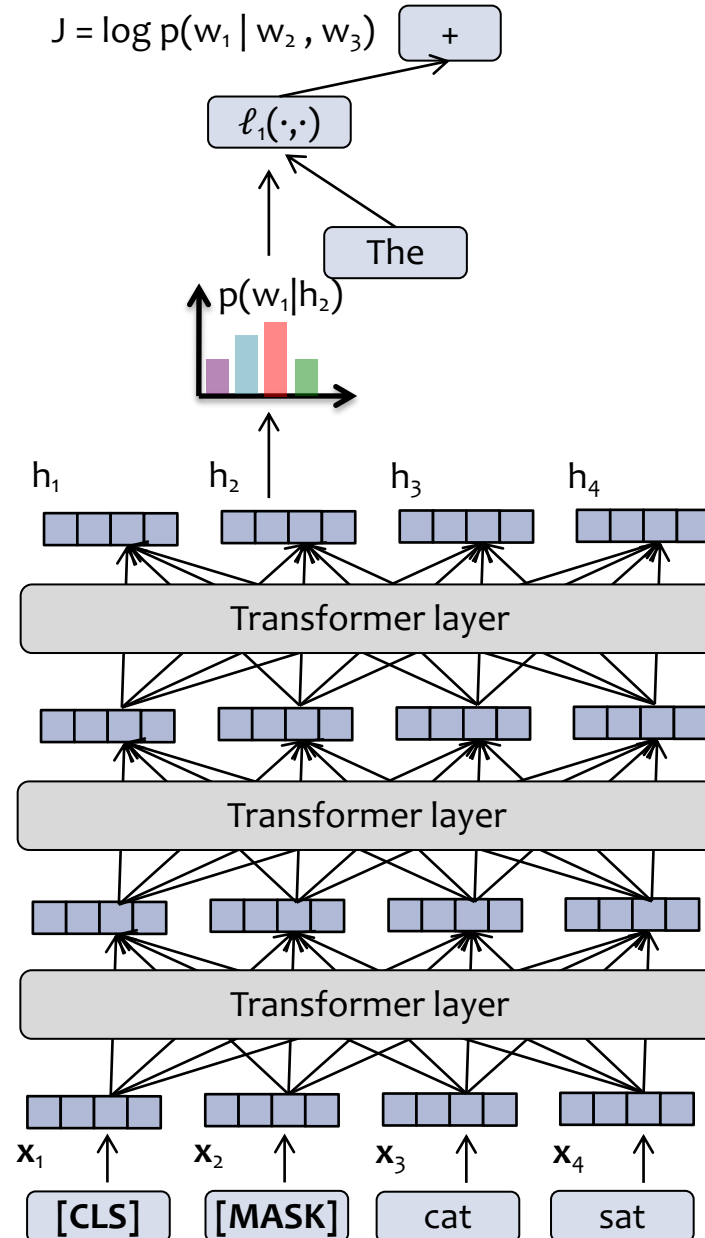
The distribution over words is used for **masked language model (MLM) pre-training** (cf. BERT)

Encoder-only Transformer

BERT popularized this encoder-only Transformer architecture and style of pretraining

MLM Pretraining:

- Rather than trying to predict the next word from the previous ones...
- ...mask out a word (or a few words) and predict the missing words from the remaining ones



Each layer of an encoder-only Transformer consists of several **sublayers**:

1. non-causal attention
2. feed-forward neural network
3. layer normalization
4. residual connections

Each hidden vector looks at the the hidden vectors of **all timesteps in the previous layer**.

The distribution over words is used for **masked language model (MLM) pre-training** (cf. BERT)

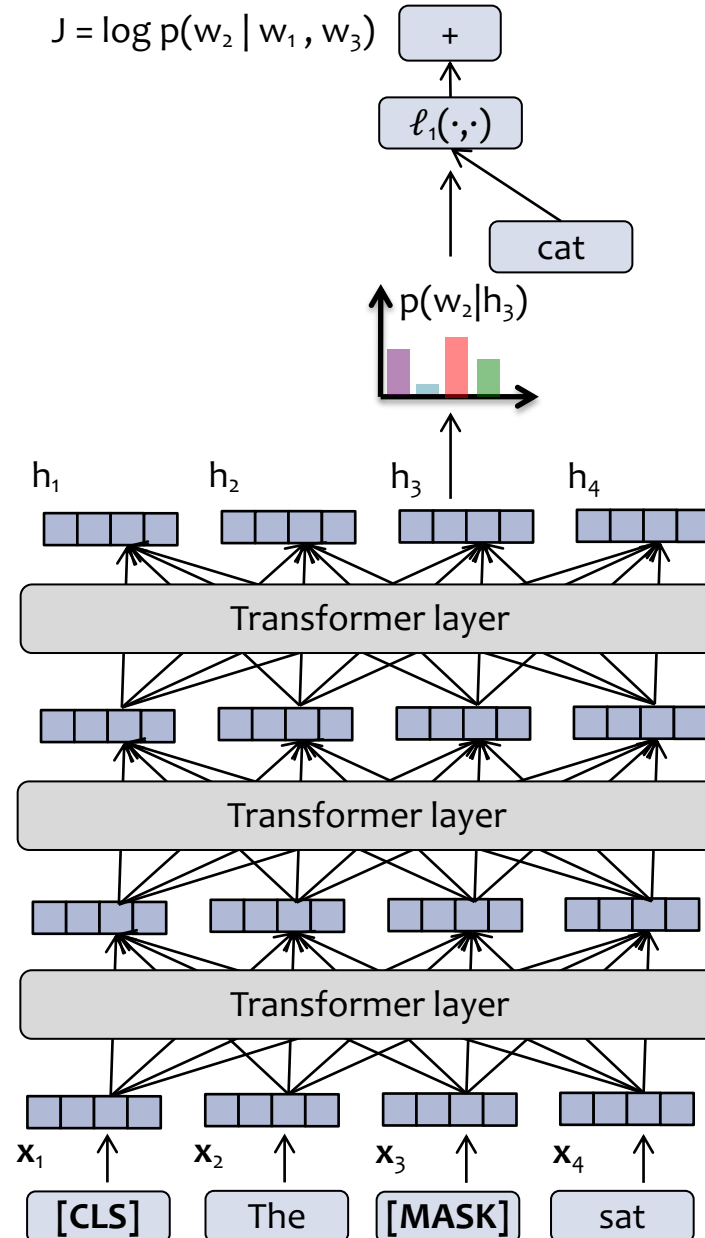
...

Encoder-only Transformer

BERT popularized this encoder-only Transformer architecture and style of pretraining

MLM Pretraining:

- Rather than trying to predict the next word from the previous ones...
- ...mask out a word (or a few words) and predict the missing words from the remaining ones



Each layer of an encoder-only Transformer consists of several **sublayers**:

1. non-causal attention
2. feed-forward neural network
3. layer normalization
4. residual connections

Each hidden vector looks at the the hidden vectors of **all timesteps in the previous layer**.

The distribution over words is used for **masked language model (MLM) pre-training** (cf. BERT)

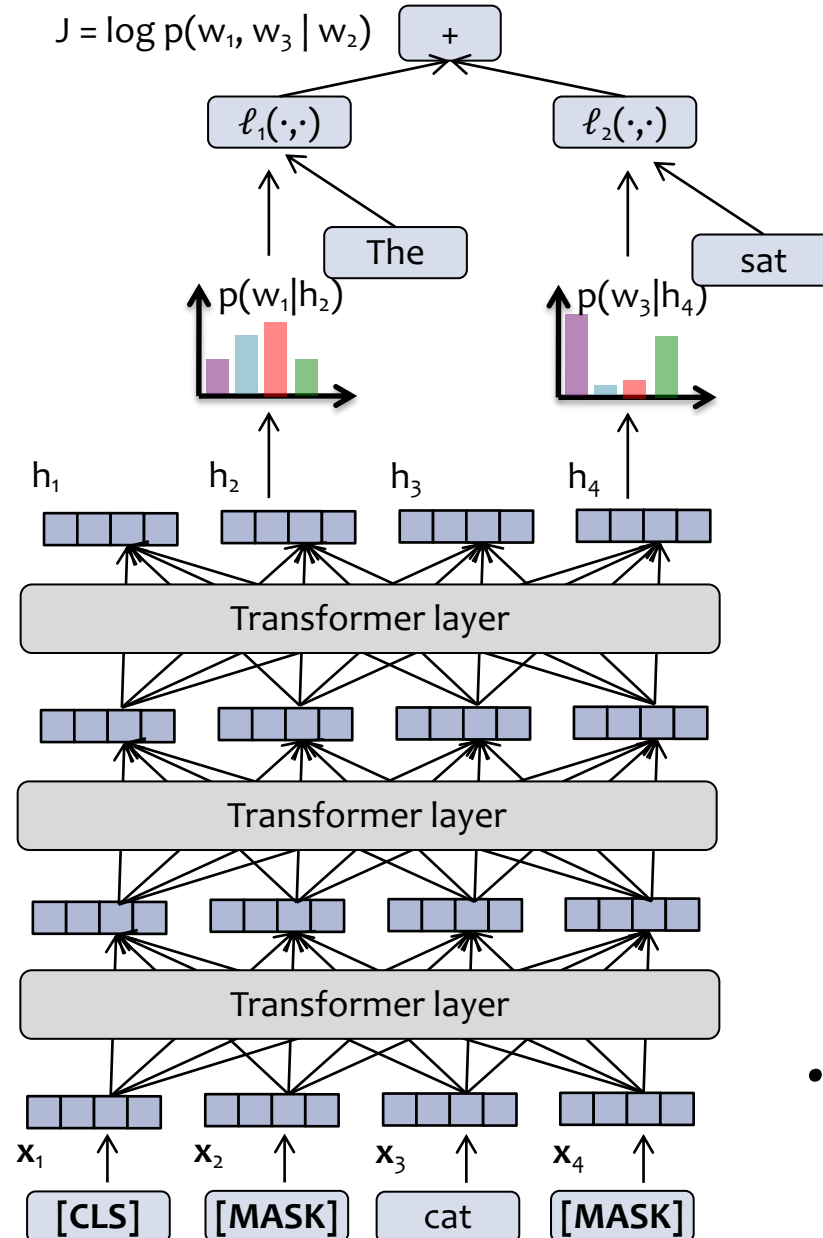
...

Encoder-only Transformer

BERT popularized this encoder-only Transformer architecture and style of pretraining

MLM Pretraining:

- Rather than trying to predict the next word from the previous ones...
- ...mask out a word (or a few words) and predict the missing words from the remaining ones



Each layer of an encoder-only Transformer consists of several **sublayers**:

1. non-causal attention
2. feed-forward neural network
3. layer normalization
4. residual connections

Each hidden vector looks at the the hidden vectors of **all timesteps in the previous layer**.

The distribution over words is used for **masked language model (MLM) pre-training** (cf. BERT)

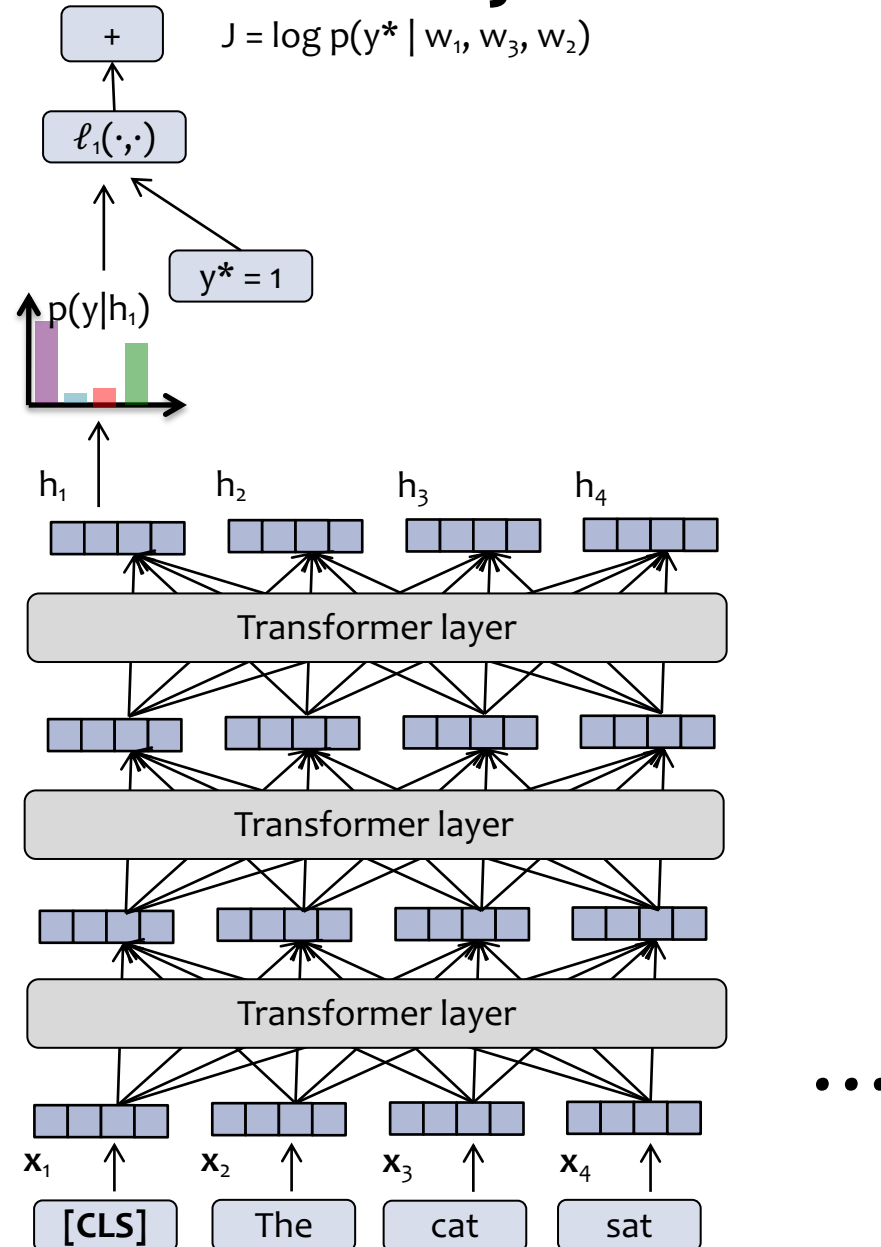
...

Encoder-only Transformer

BERT popularized this encoder-only Transformer architecture and style of pretraining

MLM Pretraining:

- Rather than trying to predict the next word from the previous ones...
- ...mask out a word (or a few words) and predict the missing words from the remaining ones



Supervised Fine-tuning:

- How to fine-tune this model for a classification task?
- Predict the class label given the embedding for the special [CLS] token
- Fine-tune the model to be good at classification in this way

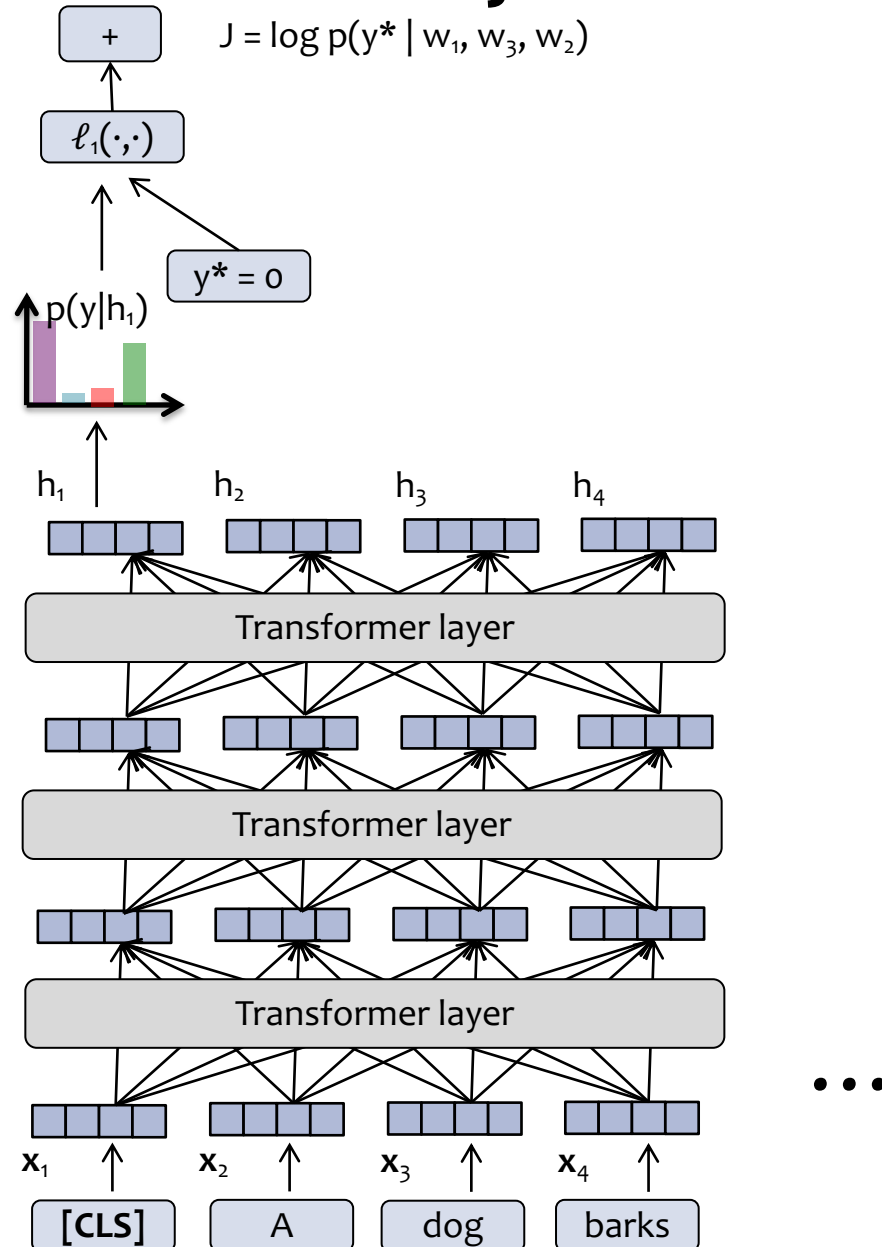
Although this is not a generative language model, it can be used very effectively as a discriminator

Encoder-only Transformer

BERT popularized this
encoder-only
Transformer
architecture and style
of pretraining

MLM Pretraining:

- Rather than trying to predict the next word from the previous ones...
- ...mask out a word (or a few words) and predict the missing words from the remaining ones



Supervised Fine-tuning:

- How to fine-tune this model for a classification task?
- Predict the class label given the embedding for the special [CLS] token
- Fine-tune the model to be good at classification in this way

Although this is not a generative language model, it can be used very effectively as a discriminator

VISION TRANSFORMER

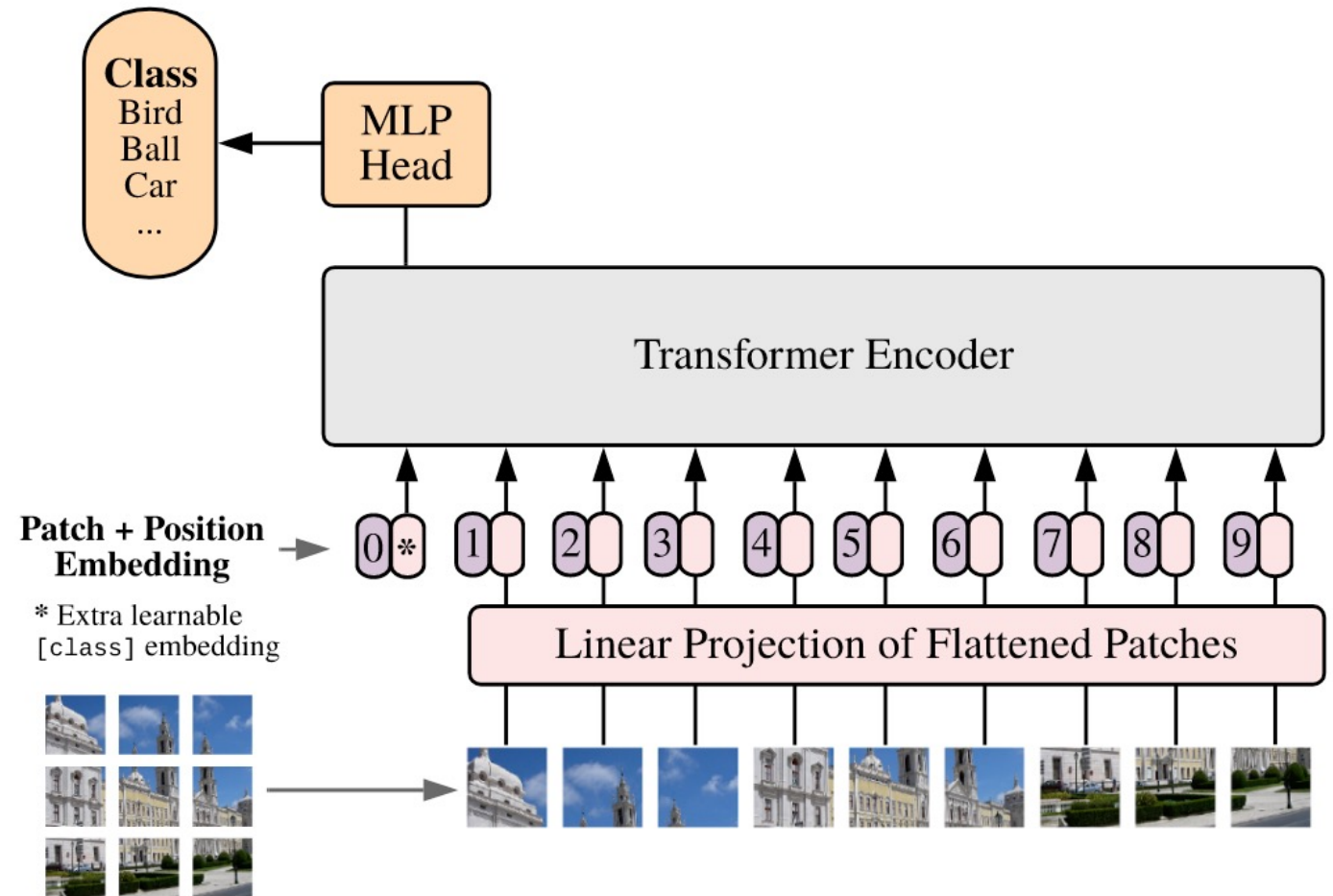
Vision Transformer (ViT)

Model:

- model is almost identical to BERT
- instead of words as input the inputs are $P \times P$ pixel image patches, $P \in \{14, 16, 32\}$ (no overlap)
- each patch is embedded linearly into a vector of size 1024
- 1D positional embeddings

Training:

- for pre-training, optimize for image classification on large supervised dataset (e.g. ImageNet 21K, JFT-300M)—same setup as a CNN
- for fine-tuning, learn a new classification head on a small dataset (e.g. CIFAR-100)



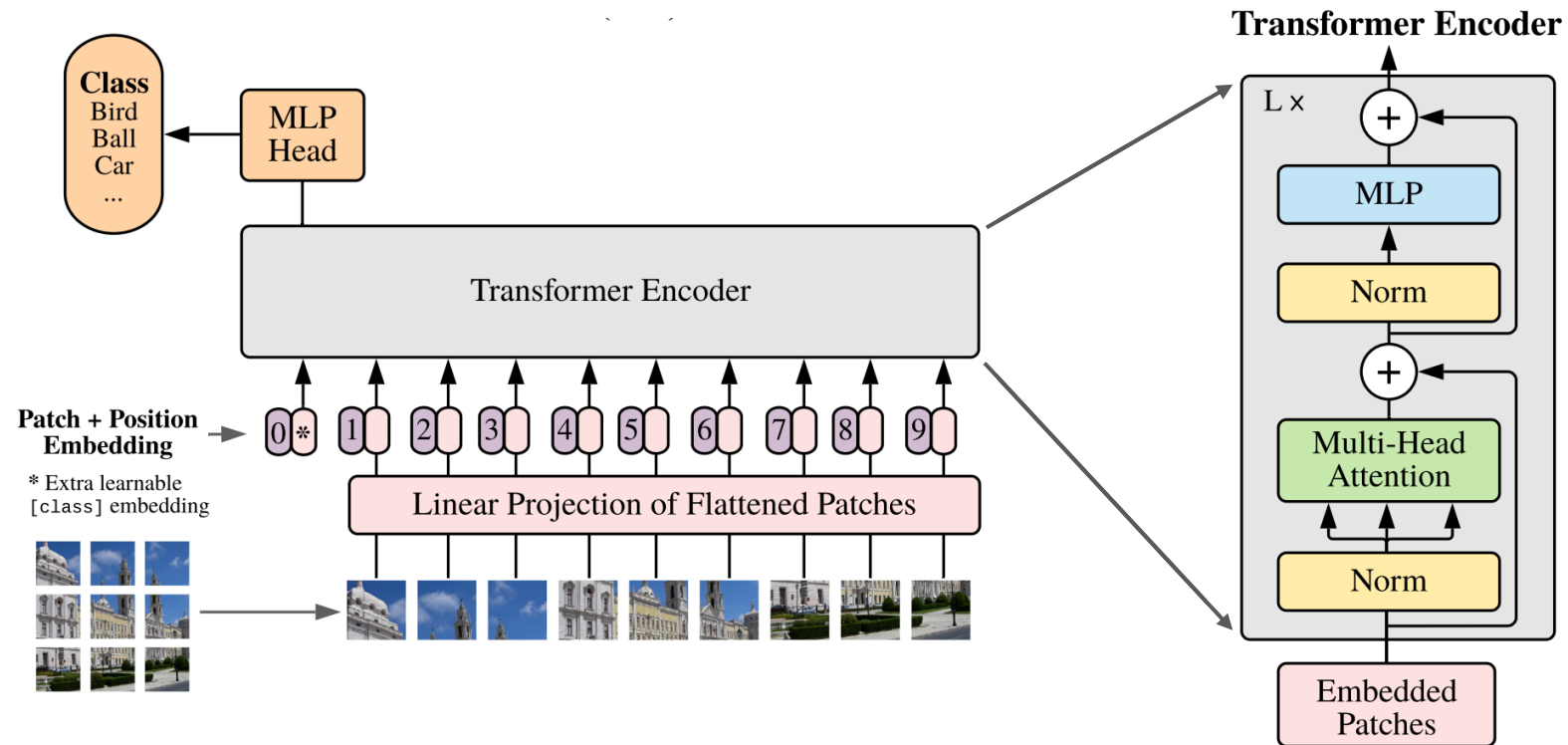
Vision Transformer (ViT)

Model:

- model is almost identical to BERT
- instead of words as input the inputs are $P \times P$ pixel image patches, $P \in \{14, 16, 32\}$ (no overlap)
- each patch is embedded linearly into a vector of size 1024
- 1D positional embeddings

Training:

- for pre-training, optimize for image classification on large supervised dataset (e.g. ImageNet 21K, JFT-300M)—same setup as a CNN
- for fine-tuning, learn a new classification head on a small dataset (e.g. CIFAR-100)



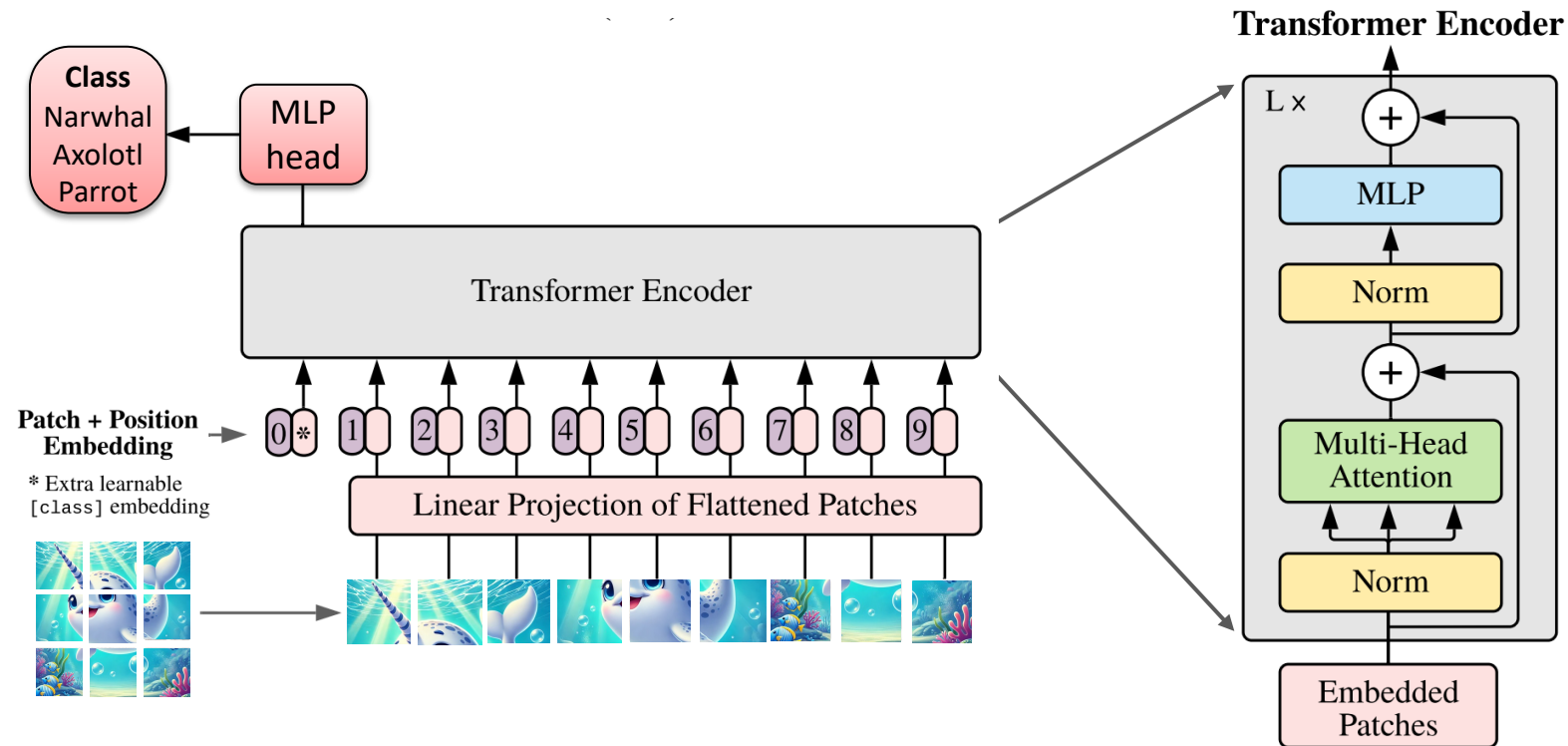
Vision Transformer (ViT)

Model:

- model is almost identical to BERT
- instead of words as input the inputs are $P \times P$ pixel image patches, $P \in \{14, 16, 32\}$ (no overlap)
- each patch is embedded linearly into a vector of size 1024
- 1D positional embeddings

Training:

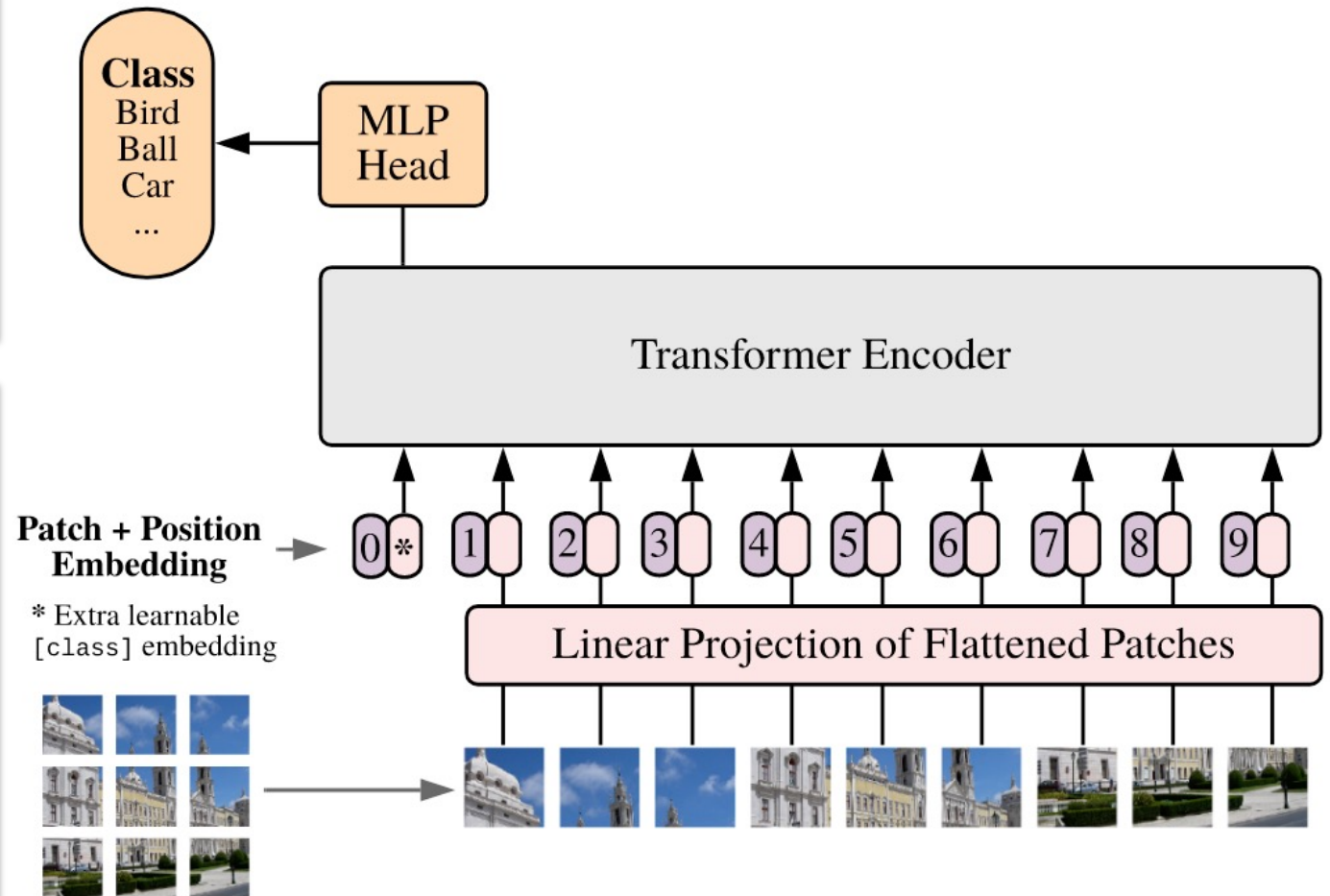
- for pre-training, optimize for image classification on large supervised dataset (e.g. ImageNet 21K, JFT-300M)—same setup as a CNN
- **for fine-tuning, learn a new classification head on a small dataset (e.g. CIFAR-100)**



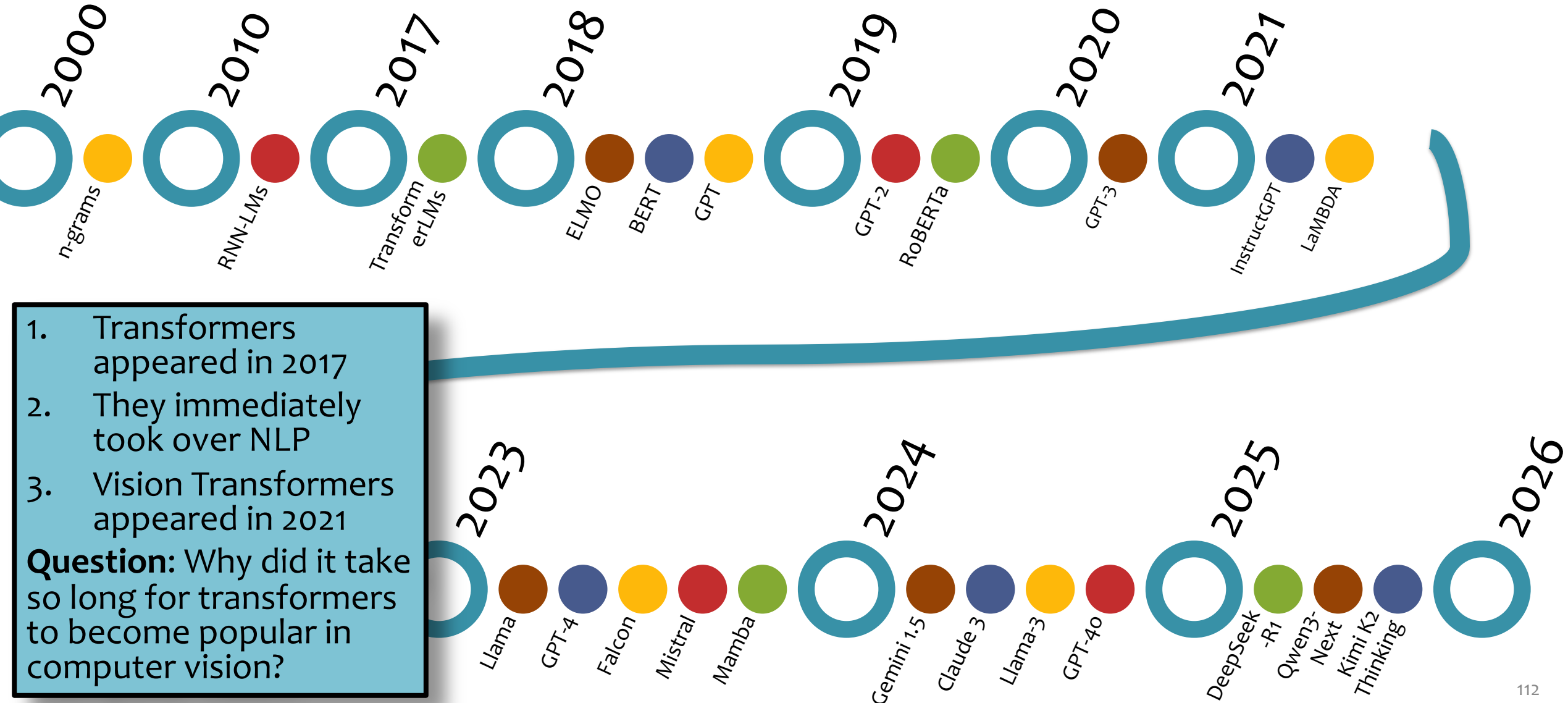
Vision Transformer (ViT)

Question: how can a ViT learn 2D positional information from 1D position embeddings?

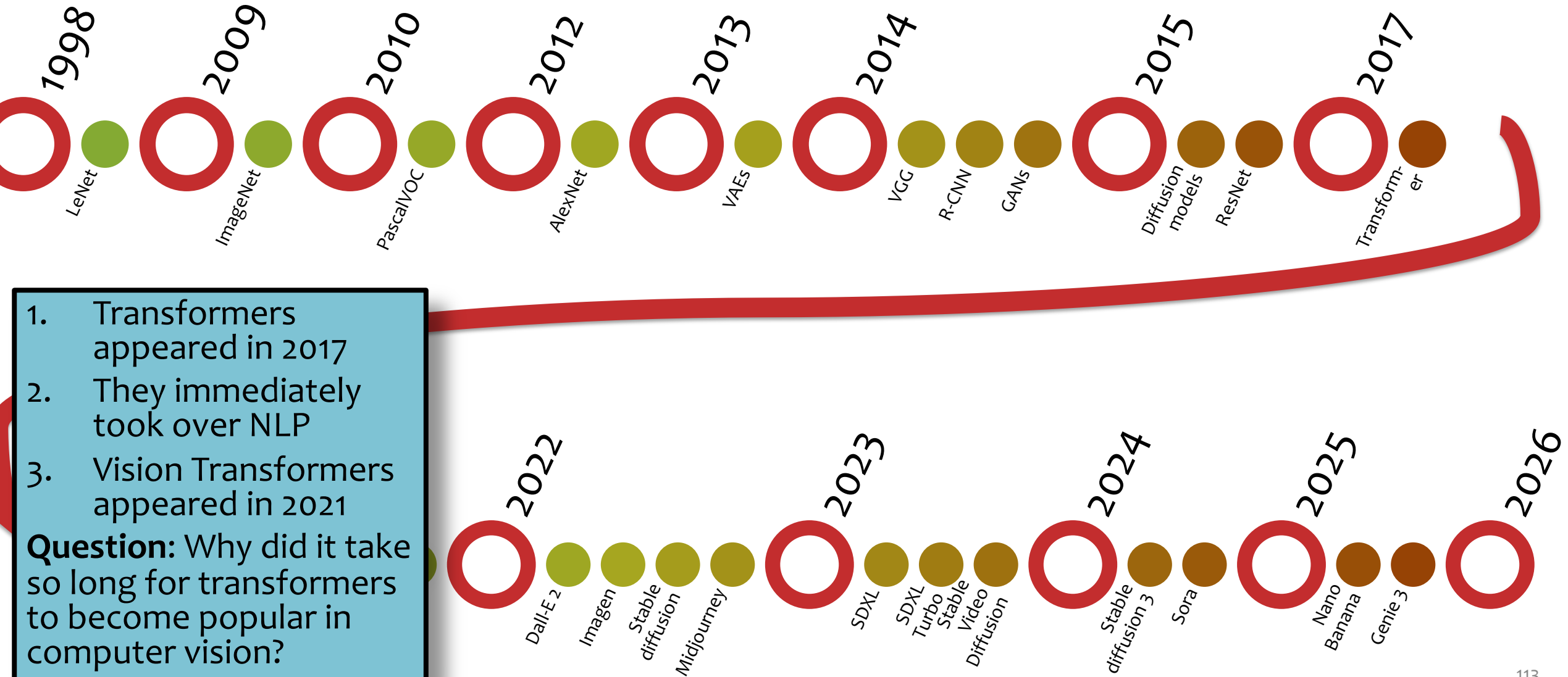
Answer:



Timeline: Language Modeling



Timeline: Image Generation



Timeline: Image Generation

AN IMAGE IS WORTH 16X16 WORDS: TRANSFORMERS FOR IMAGE RECOGNITION AT SCALE

**Alexey Dosovitskiy^{*,†}, Lucas Beyer^{*}, Alexander Kolesnikov^{*}, Dirk Weissenborn^{*},
Xiaohua Zhai^{*}, Thomas Unterthiner, Mostafa Dehghani, Matthias Minderer,
Georg Heigold, Sylvain Gelly, Jakob Uszkoreit, Neil Houlsby^{*,†}**

^{*}equal technical contribution, [†]equal advising

Google Research, Brain Team

{adosovitskiy, neilhoulby}@google.com

When trained on mid-sized datasets such as ImageNet without strong regularization, these models yield modest accuracies of a few percentage points below ResNets of comparable size. This seemingly discouraging outcome may be expected: Transformers lack some of the inductive biases inherent to CNNs, such as translation equivariance and locality, and therefore do not generalize well when trained on insufficient amounts of data.

However, the picture changes if the models are trained on larger datasets (14M-300M images). We find that large scale training trumps inductive bias. Our Vision Transformer (ViT) attains excellent results when pre-trained at sufficient scale and transferred to tasks with fewer datapoints. When pre-trained on the public ImageNet-21k dataset or the in-house JFT-300M dataset, ViT approaches or beats state of the art on multiple image recognition benchmarks. In particular, the best model reaches the accuracy of 88.55% on ImageNet, 90.72% on ImageNet-Real, 94.55% on CIFAR-100, and 77.63% on the VTAB suite of 19 tasks.

1. Transformers appeared in 2017
2. They immediately took over NLP
3. Vision Transformers appeared in 2021

Question: Why did it take so long for transformers to become popular in computer vision?

Timeline: Image Generation

1. Transformers appeared in 2017
2. They immediately took over NLP
3. Vision Transformers appeared in 2021

Question: Why did it take so long for transformers to become popular in computer vision?

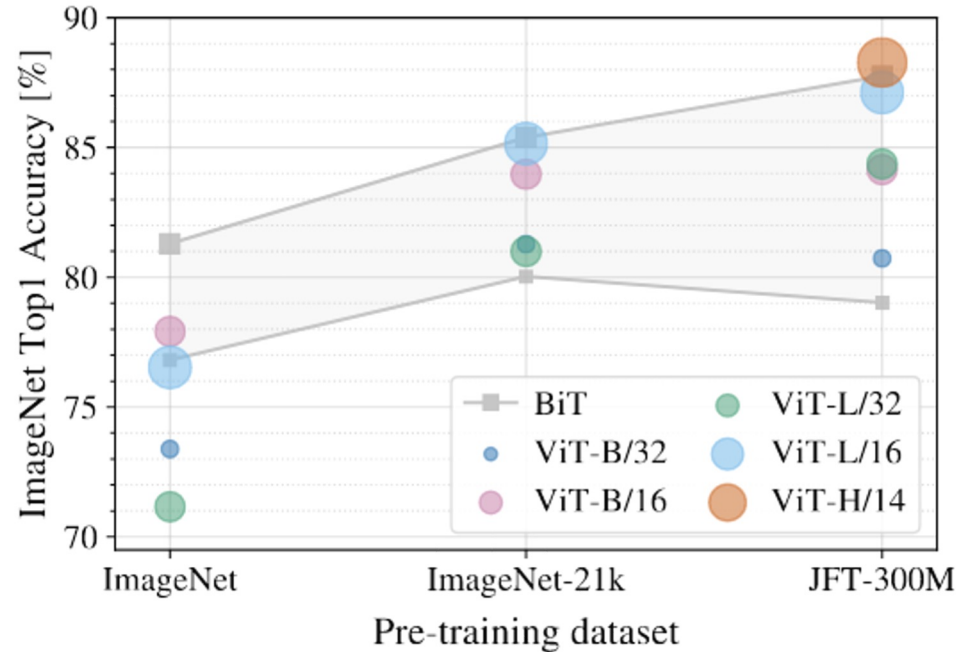


Figure 3: Transfer to ImageNet. While large ViT models perform worse than BiT ResNets (shaded area) when pre-trained on small datasets, they shine when pre-trained on larger datasets. Similarly, larger ViT variants overtake smaller ones as the dataset grows.

Comparison of two model families:

1. BiT – large CNNs based on ResNet
2. ViT – vision transformers of various sizes

Vision Transformer (ViT)

- The original Vision Transformer models were quite small compared to the Large Language Models (LLMs) of the time
- By 2023, ViT had been scaled to 22 billion parameters with good success

Model	Layers	Hidden size D	MLP size	Heads	Params
ViT-Base	12	768	3072	12	86M
ViT-Large	24	1024	4096	16	307M
ViT-Huge	32	1280	5120	16	632M

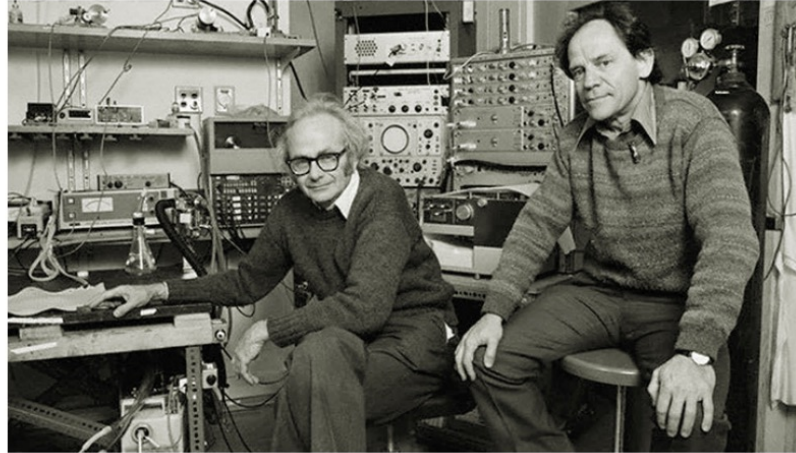
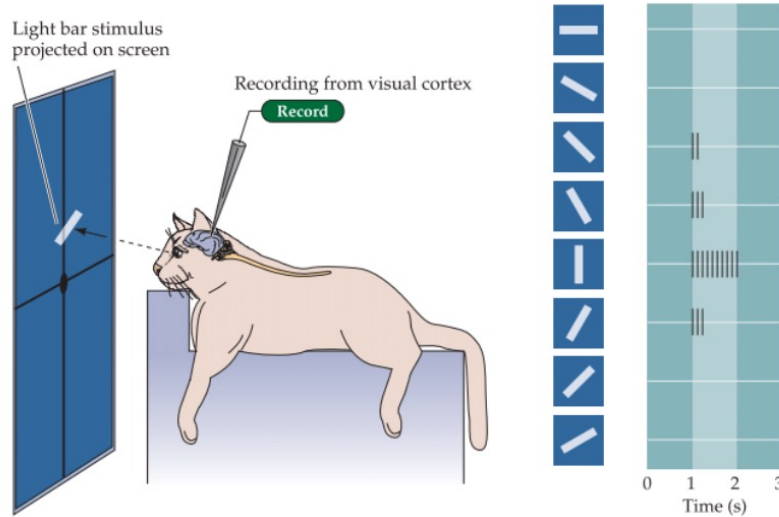
Table 1: Details of Vision Transformer model variants.

Name	Width	Depth	MLP	Heads	Params [M]
ViT-G	1664	48	8192	16	1843
ViT-e	1792	56	15360	16	3926
ViT-22B	6144	48	24576	48	21743

Table 1: ViT-22B model architecture details.

Attention's Neuroscience Origins

Re: CNN



David Hubel & Torsten Wiesel
(Nobel 1981)

Re: Attention

$$w_t^c(i) = \frac{\exp(\beta_t \cdot \langle k_t, M_t(i) \rangle)}{\sum_j \exp(\beta_t \cdot \langle k_t, M_t(j) \rangle)}$$

- β_t = coupling strength
- k_t = output from the controller (DNN)
- $\langle \cdot, \cdot \rangle$ = similarity measure e.g. cosine distance
- competition between memories

Competition between Hopfield net memories is what
we use as a softmax for the attention weights!



John Hopfield (Nobel 2024)

Hopfield nets (1982)

Given the dynamical electrochemical properties of neurons and their interconnections (synapses), we readily understand schemes that use a few neurons to obtain elementary useful biological behavior (1-3). Our understanding of such simple circuits in electronics allows us to plan larger and more complex circuits which are essential to large computers. Because evolution has no such plan, it becomes relevant to ask whether the ability of large collections of neurons to perform "computational" tasks may in part be a spontaneous collective consequence of having a large number of interacting simple neurons.